МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

КАЗАНСКИЙ ГОСУДАРСТВЕННЫЙ АРХИТЕКТУРНО-СТРОИТЕЛЬНЫЙ УНИВЕРСИТЕТ

Р.С.Хайруллин

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ

Учебное пособие

Казань
2022

УДК 004.43
ББК 32.973-018.1
Х15

Хайруллин Р.С.

Х15 Объектно-ориентированное программирование: учебное пособие. – Хайруллин Р.С. – Казань: Изд-во Казан.гос.архитект.-строит.ун-та, 2022. – 140 с.

ISBN 978-5-

Печатается по решению Редакционно-издательского совета Казанского государственного архитектурно-строительного университета

Учебное пособие содержит материал по программированию на языке C#. Здесь рассмотрены вопросы, связанные с объектно-ориентированным программированием.

Пособие предназначено для бакалавров, магистров и аспирантов, обучающихся по направлениям 09.03.02 «Информационные системы и технологии» и 09.06.01 «Информатика и вычислительная техника», а также лиц, самостоятельно изучающих язык C#.

Рецензенты:
Доктор технических наук, профессор,
заведующий кафедрой физики и математики КГАУ
Р.И.Ибятов
Кандидат технических наук, доцент,
проректор по информатизации КГАСУ
Д.М.Кордончик

УДК 004.43
ББК 32.973-018.1

© Казанский государственный архитектурно-строительный университет, 2022
© Хайруллин Р.С., 2022

ISBN 978-5-

Оглавление

Введение 5

Глава 1. Создание класса 7

§ 1.1. Назначение класса и его структура 7
§ 1.2. Поля и свойства 8
§ 1.3. Методы 12
§ 1.4. Описание класса Human и его использование 13
§ 1.5. Конструкторы класса 17
§ 1.6. Классы как поля других классов 20
§ 1.7. Вложенные классы 23
§ 1.8. Перегрузка операций 25
§ 1.9. Массивы пользовательских объектов 27
§ 1.10. Структуры 28
§ 1.11. Частичные классы и методы 33
§ 1.12. Анонимные типы 36

Глава 2. Дочерние классы 38

§ 2.1. Наследование 38
§ 2.2. Полиморфизм 40
§ 2.3. Конструктор дочернего класса 41
§ 2.4. Запись экземпляра дочернего класса в экземпляр родительского класса 42
§ 2.5. Виртуальные методы 44
§ 2.6. Класс System.Object 45
§ 2.7. Абстрактные классы 50
§ 2.8. Интерфейсы 55
§ 2.9. Использование классов для хранения данных 59

Глава 3. Обобщение 65

§ 3.1. Поля для хранения данных неизвестного типа 65
§ 3.2. Обобщенные классы 69
§ 3.3. Использование нескольких универсальных параметров 72
§ 3.4. Использование обобщенного класса при типизации 74
§ 3.5. Использование обобщенных типов для статических полей 76

§ 3.6. Обобщенные методы 77
§ 3.7. Наследование обобщенных классов 78
§ 3.8. Ограничения обобщений 84

Глава 4. Индексаторы 90

§ 4.1. Описание индексатора 90
§ 4.2. Перегрузка индексатора 92
§ 4.3. Индексирование компонентов поля 94

Глава 5. Коллекции 98

§ 5.1. Список List<T> 98
§ 5.2. Очередь Queue<T> 103
§ 5.3. Стек Stack<T> 106
§ 5.4. Словарь Dictionary<K,V> 110

Глава 6. Делегаты и события 114

§ 6.1. Понятие делегата 114
§ 6.2. Добавление методов в делегат и их удаление из делегата 116
§ 6.3. Использование делегатов при описании методов 122
§ 6.4. Анонимные методы и лямбда-выражения 123
§ 6.5. Делегаты как поля классов 126
§ 6.6. События 130
§ 6.7. Делегат EventHandler 135
§ 6.8. Встроенные делегаты 138

Введение

В настоящее время в компьютерных науках большой популярностью пользуется объектный подход. Его суть состоит в том, что вместо работы с отдельными типами данных (числами, строками и т.п.) работают с объектами, которые обладают определенными характеристиками и способны выполнять определенные функции. Здесь ключевым понятием является класс, а объекты - это экземпляры некоторых классов. Программа представляет собой некоторую совокупность взаимодействующих между собой классов.

Более того, на основе одних классов можно создавать другие, дочерние, классы. Это позволяет существенно ускорить разработку новых программ, так как имеется возможность использование ранее созданных классов.

Квалификация специалистов по информационным технологиям часто оценивается по умению создавать и работать с разными видами классов при разработке соответствующих приложений. Поэтому является актуальным изучение всех этих вопросов при подготовке бакалавров по направлению 09.03.02 «Информационные системы и технологии».

Данное учебное пособие содержит материал по разработке на C# классов различных видов и их использованию. Пособие может быть использовано в качестве основной или дополнительной литературы при изучении дисциплин «Объектно-ориентированное программирование», «Языки программирования», «Технология программирования», а также при выполнении курсовых и выпускных работ бакалаврами, обучающимися по направлению 09.03.02 «Информационные системы и технологии».

Книга состоит из шести глав. Первая глава посвящена основам разработки классов. Здесь подробно разобраны принципы описания различных членов класса, в том числе и статических.

Во второй главе изложены методы создания дочерних классов. Здесь также рассмотрены вопросы использования классов для хранения данных.

В третьей главе изложен материал, связанный с обобщенными типами. Здесь рассмотрены обобщенные классы и методы, наследование обобщений и ограничения на обобщения.

Четвертая глава посвящена индексаторам. Изложены методы их описания и применения.

В пятой главе собран материал по коллекциям. Рассмотрены списки, очереди, стеки и словари.

В последней главе рассмотрены вопросы, связанные с делегатами и событиями. Здесь большое внимание уделено использованию делегатов, в том числе и при создании классов с событиями.

В качестве иллюстрации излагаемого материала приведено большое количество разных примеров с соответствующими программными кодами. При подготовке кодов использована среда Microsoft Visual Studio Proffecional 2015.

При составлении пособия использовались материалы, выставленные в открытом доступе на сайтах www.intuit.ru и metanit.com, в частности,

1. Биллиг В. Основы программирования на C# / Интернет-ресурс / URL: http://www.intuit.ru/studies/courses/2247/18/info
2. Кудрина Е., Огнева М., Портенко М. Программирование на языке C#: разработка консольных приложений / Интернет-ресурс / URL: http://www.intuit.ru/studies/courses/486/342/info
3. Павловская Т. Программирование на языке высокого уровня C# / Интернет-ресурс / URL: http://www.intuit.ru/studies/courses/629/485/info
4. Полное руководство по языку программирования C# 11 и платформе .NET 7 / Интернет-ресурс / URL: https://metanit.com/sharp/tutorial/
5. Столбовский Д. Разработка Web-приложений ASP .NET с использованием Visual Studio .NET / Интернет-ресурс / URL: http://www.intuit.ru/studies/courses/1139/250/info

Негосударственное образовательное частное учреждение дополнительного профессионального образования «Национальный Открытый Университет «ИНТУИТ» - это образовательный проект, главными целями которого являются свободное распространение знаний во Всемирной Сети и предоставление услуг дистанционного обучения.

Сайт metanit.com посвящен различным языкам и технологиям программирования, компьютерам, мобильным платформам и ИТ-технологиям.

Также использовалась справочная информация, содержащаяся с самой системе Visual Studio.

Глава 1. Создание класса

§ 1.1. Назначение класса и его структура

Класс является основным понятием языков объектно-ориентированного программирования, к которым относится и C#. В C# нельзя создать отдельно стоящую процедуру или функцию, она обязательно должна входить в некоторый класс. Аналогично, любая переменная должна быть объявлена в некотором классе, причем тип сам описывается соответствующим классом. Мы всегда работаем в пределах одного класса и при этом используем ресурсы других классов: либо создаем экземпляры этих классов и работаем с ними, либо используем статические свойства и методы без создания экземпляров класса.

Все современные программные системы создаются с использованием принципов объектно-ориентированного программирования. Это означает, что любая сущность, с которой осуществляются какие-либо действия, должна быть описана в виде класса. Класс - это некоторый шаблон, который определяет свойства и методы объектов, создаваемых как экземпляры данного класса. Основное назначение класса - создать некий абстрактный тип данных, характерный для решаемой задачи.

Таким образом, программная система представляет собой совокупность взаимодействующих друг с другом классов.

Предположим, что есть некоторый объект, характеризующийся определенными параметрами, например, сотрудник фирмы. У него есть фамилия, имя, отчество, возраст и т. п. В таких случаях удобно каждого сотрудника описывать не рядом независимых переменных (строкового типа для фамилии, имени и отчества, целого типа для возраста), а одной переменной, например, типа `Human`, внутри которой и содержаться переменные для фамилии, имени, отчества и возраста. Этот факт здесь самый важный, что в переменной типа `Human` содержатся другие переменные. Типа `Human` нет среди встроенных типов данных, но его можно ввести. Для этого следует создать соответствующий класс `Human`. Этот код обычно включает в себя следующие фрагменты:

- описание полей;
- описание свойств;
- описание методов.

Описанные здесь поля, свойства и методы называются членами класса. Могут быть и другие члены класса. Обычно члены связаны с отдельными экземплярами класса. Поля предназначены для хранения

данных экземпляра, свойства и методы – для работы с ними. Данные могут быть связаны с самим классом, например, количество созданных экземпляров этого класса. Для работы с ними могут использоваться соответствующие свойства и методы. Такие члены класса называются статическими. При их описании используется ключевое слово `static`. Для простоты, члены, не являющиеся статическими, мы будем называть динамическими.

§ 1.2. Поля и свойства

Поле класса (переменная-член) — переменная, связанная с классом или объектом. Все данные объекта хранятся в его полях. Доступ к полям осуществляется по их имени. Обычно тип данных каждого поля задаётся в описании класса, членом которого является поле. Как правило, поля объявляются закрытыми, т.е. используется модификатор доступа `private`. При необходимости значение поля можно инициализировать непосредственно в теле класса. Имена полей принято записывать со строчной буквы.

Излагаемый материал мы будем иллюстрировать при примере описании класса `Human`, который в дальнейшем будем использовать. Этот класс предназначен для хранения и обработки информации о сотрудниках некоторой фирмы. В качестве полей будем использовать фамилию, имя, отчество и возраст сотрудника, которые назовем `lastname`, `name`, `secondname`, `age`. Также опишем статическое поле `counthuman` для хранения количества созданных экземпляров класса и приведем пример инициализации поля при его описании.

Отметим, что в примерах для простоты мы будем использовать консольные приложения, хотя это легко переносится на приложения других видов.

Итак, имеем

```
// описание полей
private string name;
private string secondname;
private string lastname;
private int age;
static private int counthuman;
private int basis = 1000;
```

Свойства – это средства доступа к полям, позволяющие считывать и изменять его данные. Чтобы использовать свойства для изменения значений полей и их чтения, нужно описать, соответственно, функции `set`

и get. Эти функции должны быть открытыми, т.е. иметь модификатор доступа public. Функция set присваивает значение value некоторому полю. При этом может, например, выполняться проверка допустимости вводимого значения. Функция get возвращает некоторый результат, который получается на основе значений полей, а может быть просто значением некоторого поля. Этот вариант чаще всего и используется. Принято свойствам давать имена, совпадающие с именем соответствующего поля и начинающиеся с прописной буквы.

Если поля объявить открытыми, т.е. использовать модификатор доступа public, то можно будет читать и изменять их значения непосредственно присваиванием из методов других классов. В этом случае необходимость описания свойств отпадает. Но это нецелесообразно. Так как именно свойства позволяют регулировать чтение или изменение поля, проверять допустимость новых значения полей и т.п. Необязательно должны присутствовать обе функции и set, и get. Например, свойство может быть только для чтения, тогда описывается только функция get. Приведем примеры.

```
// описание свойств
public string SecondName
{
    set
    {
        secondname = value;
    }
}
public string LastName
{
    set
    {
        lastname = value;
    }
}
public string Name
{
    set
    {
        name = value;
    }
}
public int Age
{
```

```
    get
    {
        return age;
    }
    set
    {
        age = value;
    }
}
public string FullName
{
    get
    {
        return lastname + " " + name + " " + secondname;
    }
}
static public int CountHuman
{
     set
     {
         counthuman = value;
     }
}
```

Здесь мы описали свойства для установки значений полей `lastname`, `name`, `secondname`, статического поля `counthuman`. Для поля же `age` описали свойства, как для установки, так и для чтения. Кроме того, описали свойство `FullName`, которое не соответствует конкретному полю и выводит значение, формируемое из значений трех полей.

Можно установить модификатор доступа на одну из функций, например,

```
public int Age
{
    get
    {
        return age;
    }
    private set
    {
        age = value;
    }
```

```
}
```

В этом случае блок `set` можно будет использовать только в этом классе.

Существуют автоматические свойства, в которых функции `set` и `get` записываются без тела, например,

```
public int Age { get; set;}
```

В этом случае поле описывать не нужно. Оно создается автоматически и по имени совпадает со свойством. Здесь также можно использовать модификатор

```
public int Age { get; private set;}
```

и создавать поля только для чтения

```
public int Age { get;}
```

Во всех вышеприведенных примерах мы использовали минимальную функциональность свойств. Они только позволяли установить значение поля или считать его. Однако, как уже указывалось, здесь могут присутствовать и другие операторы. Предположим, что в поле `basis` содержится конфиденциальная информация и его чтение допускается только после ввода пароля. Тогда свойство может иметь вид

```
public int Basis
{
     get
     {
          Console.WriteLine("Введите пароль");
          string pw = Console.ReadLine();
          if (pw == "123")
             return basis;
          else throw new Exception("Неправильный
                пароль!");
      }
}
```

Теперь при выполнении кода, когда появится запрос на значение поля `basis`, появится предложение ввести пароль. Только в случае правильного ввода пароля «123» требуемые данные будут считаны, в противном случае

будет создано новое исключение, а именно выполнение программы прервется и появится окно с приведенным текстом.

§ 1.3. Методы

Методы представляют собой процедуры и функции. Они могут быть динамическими, связанными с некоторыми объектами, и статическими. Если метод используется только внутри самого класса, где он описан, то этот метод можно объявить закрытым. Если же предполагается его использование в методах другого класса, то метод необходимо объявить открытым.

В теле динамических методов могут использоваться, как динамические члены класса, так и статические. В то время как в теле статических методов могут использоваться только статические члены.

Для использования динамических методов в некотором методе другого класса нужно создать экземпляр рассматриваемого класса и для этого экземпляра применить метод. Можно сказать, что такие методы определяют поведение объекта.

Для использования статических методов создание экземпляра не требуется, и при их вызове следует указывать имя класса, где метод описан.

Можно использовать методы для чтения или изменения значений полей, вместо свойств. В этом случае придется описать одну функцию для чтения и другую – для изменения. Поэтому проще использовать именно свойства.

Рассмотрим примеры методов. Опишем два метода. Первый метод `Info()` – открытый динамический метод, который предназначен для вывода информации о сотруднике на консоль. Второй метод `Option()` - закрытый динамический метод, который определяет варианты записи возраста в зависимости от количества лет и передает их методу `Info()`.

```
// описание методов
public void Info()
{
     Console.WriteLine(lastname + " " + name + " " +
       secondname + "," + age + Option());
}

private string Option()
{
    string s;
```

```
    if(age > 10 && age <= 20)
        s = " лет";
    else
    {
        switch(age % 10)
        {
            case 1: s = " год";
               break;
            case 2:
            case 3:
            case 4: s = " года";
               break;
            default: s = " лет";
               break;
        }
    }
    return s
}
```

§ 1.4. Описание класса Human и его использование

Приведем пример описания и использования класса Human. Выше мы рассматривали разные фрагменты кода в качестве примеров. Сейчас на их основе создадим класс Human. Этот класс мы будем дополнять по мере изучения материала. Пока опишем поля lastname, name, secondname, age и свойства LastName, Name, SecondName, Age, FullName. Последнее свойство защитим паролем. Также опишем методы Info() и Option(), которые будут использоваться для вывода на консоль данных о сотруднике.

Чтобы создать класс нужно выбрать пункт меню Проект -> Добавить класс. В открывшемся окне выбрать элемент Класс и ввести имя класса, в данном случае Human. По умолчанию будут использоваться имена Class1 и т.д. После этого следует нажать кнопку Добавить. В результате будет создан файл с указанным именем и расширением cs, и соответствующая вкладка добавлена в рабочей области окна. На этой вкладке мы получим шаблон вида

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
```

```
using System.Threading.Tasks;

namespace Project1
{
    class Human
    {
    }
}
```

Здесь `Project1` – имя пространства имен, в которое войдет наш класс `Human`. Теперь надо во внутренних скобках записать код, который может иметь следующий вид:

```
// описание полей
private string name;
private string secondname;
private string lastname;
private int age;
// описание свойств
public string SecondName
{
    set
    {
        secondname = value;
    }
}
public string LastName
{
    set
    {
        lastname = value;
    }
}
public string Name
{
    set
    {
        name = value;
    }
}
public int Age
{
```

```
    get
    {
        return age;
    }
    set
    {
        age = value;
    }
}
public string FullName
{
    get
    {
        Console.WriteLine("Введите пароль");
        string pw = Console.ReadLine();
        if (pw == "456")
            return lastname + " " + name + " " +
secondname;
        else
            throw new Exception("Неправильный пароль!");
    }
}

// описание методов
public void Info()
{
    Console.WriteLine(lastname + " " + name + " " +
    secondname + "," + age + Option());
}

private string Option()
{
    string s;
    if (age > 10 && age <= 20)
        s = " лет";
    else
    {
        switch (age % 10)
        {
             case 1:
                s = " год";
```

```
                break;
            case 2:
            case 3:
            case 4:
                s = " года";
                break;
            default:
                s = " лет";
                break;
        }
    }
    return s;
}
```

Теперь можно использовать этот класс. Для этого в процедуру `Main()` класса `Program` запишем следующий код

```
Human hmn1 = new Human();
hmn1.LastName = "Иванов";
hmn1.Name = "Василий";
hmn1.SecondName = "Петрович";
hmn1.Age = 45;
hmn1.Info();
Human hmn2 = new Human();
hmn2.LastName = "Сидоров";
hmn2.Name = "Александр";
hmn2.SecondName = "Михайлович";
hmn2.Age = 42;
Console.WriteLine("Сотрудник {0}, возраст {1}",
      hmn2.FullName, hmn2.Age);
```

Здесь мы создали объекты `hmn1`, `hnm2`, полям этих объектов присвоили соответствующие значения. Причем данные по первому объекту вывели непосредственно в процедуре `Main()`, а для вывода данных по второму объекту использовали метод `Info()`.

Заметим, что при выводе данных по второму объекту использовано свойство `FullName`. Поэтому после запуска программы мы получим строки

```
Иванов Василий Петрович, 45 лет
Введите пароль
```

Мы должны ввести пароль «456». После его ввода получим

```
Сотрудник Сидоров Александр Михайлович, возраст 42
```

§ 1.5. Конструкторы класса

Рассмотрим подробнее оператор

```
Human hmn1 = new Human();
```

Его можно разбить на два оператора

```
Human hmn1;
hmn1 = new Human();
```

Первый из них предназначен для описания переменной `hmn1` типа `Human,` а второй представляет собой вызов функции `Human()`, результат выполнения которой присваивается переменной `hmn1.` Эта функция называется конструктором данного класса, и ее назначение – создание нового экземпляра этого класса. Здесь мы использовали конструктор по умолчанию.

Как видим, имя конструктора совпадает с именем класса, и конструктор по умолчанию не имеет параметров. Конструктор – это метод класса. Как любой метод, его можно перегрузить, т.е. создать другие реализации конструктора, отличающиеся количеством параметров или их типом. Количество перегрузок может быть любым. Обычно параметры используются для инициализации всех или части полей класса. Описание конструктора аналогично описанию других методов. Единственное отличие – его имя совпадает с именем класса и у него не определен тип.

Добавим в класс конструктор

```
public Human(string lastname, string name, string
        secondname, int age)
{
    this.lastname = lastname;
    this.name = name;
    this.secondname = secondname;
    this.age = age;
}
```

В описании конструктора мы использовали имена формальных параметров, совпадающие с именами полей. Поэтому, чтобы их отличить было необходимо к именам полей добавить слово `this.`, которое означает, что это член текущего класса, в данном случае мы имеем поле

класса. Можно было для формальных параметров использовать другие имена. Тогда необходимость в слове `this.` отпадает.

Заметим, что конструктор без параметров автоматически создается только при отсутствии других конструкторов. Если мы введем конструктор с параметрами, то конструктор без параметров автоматически создаваться не будет, и при его необходимости будет нужно описать его реализацию в виде

```
public Human()
{
}
```

Теперь, после добавления в код класса описаний конструкторов с параметрами и без параметров, первые пять операторов в процедуре `Main()`

```
Human hmn1 = new Human();
hmn1.LastName = "Иванов";
hmn1.Name = "Василий";
hmn1.SecondName = "Петрович";
hmn1.Age = 45;
```

можно заменить одним

```
Human hmn1 = new Human("Иванов","Василий","Петрович",45);
```

После выполнения кода мы получим тот же самый результат, который указан выше.

В новом варианте программы мы для создания объекта `hmn1` используем конструктор с параметрами, а для создания объекта `hmn2` – конструктор без параметров.

Конструкторы могут быть и статическими. Такой конструктор не имеет параметров и срабатывает только один раз при первом обращении к классу. Он может, например, использоваться для инициализации статических полей.

Рассмотрим пример. В классе `Human` добавим статическое поле

```
static private int counthuman;
```

для хранения количества созданных экземпляров, статическое свойство для инициализации статического поля

```
static public int CountHuman
{
     set
     {
         counthuman = value;
     }
}
```

статический метод для вывода значения статического поля на консоль

```
static public void GetCountHuman()
{
    Console.WriteLine("количество Human = " + counthuman);
}
```

и статический конструктор, который будет выводить сообщение об использовании класса Human,

```
static  Human()
{
  Console.WriteLine("Вы используете класс Human");
}
```

Кроме того, в коды обоих конструкторов добавим строку

```
counthuman++;
```

для изменения значения статического поля при создании нового экземпляра.

И наконец, в Main() запишем код

```
Human.CountHuman = 0;
Human.GetCountHuman();
Human hmn1 = new Human("Иванов","Василий","Петрович",45);
Human.GetCountHuman();
hmn1.Info();
Human hmn2 = new Human();
Human.GetCountHuman();
hmn2.LastName = "Сидоров";
hmn2.Name = "Александр";
hmn2.SecondName = "Михайлович";
```

```
hmn2.Age = 42;
Console.WriteLine("Сотрудник {0}, возраст {1}",
       hmn2.FullName, hmn2.Age);
```

При выполнении кода получим строки

```
Вы используете класс Human
количество Human = 0
количество Human = 1
Иванов Василий Петрович, 45 лет
количество Human = 2
Введите пароль
456
Сотрудник Сидоров Александр Михайлович, возраст 42
```

Мы видим, что после создания каждого нового экземпляра их количество на единицу увеличивается.

§ 1.6. Классы как поля других классов

При описании полей классов мы использовали встроенные типы. Естественно можно использовать и типы, определенные пользователем. Сейчас мы рассмотрим вопросы создания классов, у которых поля сами являются экземплярами ранее нами созданных классов.

Отметим, что нет принципиальной разницы, используем ли мы встроенные типы или определенные пользователем. Необходимо только учитывать, что типы, определенные пользователем, относятся ссылочным. Поэтому при инициализации такого поля приходится использовать конструктор соответствующего класса. Этот конструктор может использоваться либо при создании класса, либо в вызывающей процедуре, например, `Main()`. Рассмотрим пример, в котором будут присутствовать оба варианта.

Создадим класс `Office` для обозначения подразделений вуза. В качестве полей будем использовать `name` – название подразделения, `countworker` – количество работающих, `boss` – информация о руководителе, причем `boss` объявим типа `Human` и воспользуемся свойствами и методами этого класса. Создадим два конструктора. Первый конструктор – без параметров. Для инициализации полей создадим соответствующие свойства. Второй конструктор – с шестью параметрами.

Также опишем метод `Info()` для вывода информации на консоль, причем данные о руководителе будем выводить с помощью метода `Info()` класса `Human`, а название подразделения и количество работающих выведем непосредственно.

Итак, получим следующий код для класса

```
class Office
{
    //описание полей
    private Human boss;
    private string name;
    private int countworker;

    //описание свойств
    public string Name
    {
        set
        {
            name = value;
        }
    }
    public int CountWorker
    {
        set
        {
            countworker = value;
        }
    }
    public Human Boss
    {
        set
        {
            boss = value;
        }
    }

    //описание метода
    public void Info()
    {
        Console.WriteLine("Подразделение: {0}, работающих
              {1} человек", name, countworker);
```

```
        boss.Info();
    }

    //описание конструкторов
    public Office()
    {
    }
    public Office(string lastname, string name, string
       secondname, int age, string name1, int countworker)
    {
        this.name = name1;
        this.countworker = countworker;
        boss = new Human(lastname, name, secondname, age);
    }
}
```

Как видим, при создании второго конструктора `Office` для инициализации поля `boss` мы использовали конструктор `Human`.

Теперь в процедуру `Main()` запишем код

```
Console.WriteLine("\nКлассы как поля");
Office off1 = new Office();
off1.Name = "кафедра  математики";
off1.CountWorker = 12;
off1.Boss = new Human("Иванов", "Петр", "Николаевич", 46);
off1.Info();
Office off2 = new Office("Алексеев", "Семен",
            "Михайлович", 53, "кафедра физики", 15);
off2.Info();
```

Здесь мы объект `off1` создали с помощью конструктора без параметров, и нам пришлось для инициализации свойства `Boss` использовать конструктор `Human`. Объект `off2` создали с помощью второго конструктора.

После выполнения кода получим

```
Классы как поля
Подразделение: кафедра математики, работающих 12 человек
Иванов Петр Николаевич, 46 лет
Подразделение: кафедра физики, работающих 15 человек
Алексеев Семен Михайлович, 53 года
```

Заметим, что классы Human и Office имеют одноименные поля name, свойства Name и методы Info(). Однако это не вызывает проблем. Только при описании конструктора класса Office с параметрами нам пришлось для обозначения параметров использовать имена name и name1.

§ 1.7. Вложенные классы

Выше мы использовали класс Human для создания класса Office. Может оказаться, что класс Human в других кодах не используется. В таких случаях удобнее описание вспомогательного класса Human поместить в тело класса Office, который его использует. Получим вложенный класс. Вложенный класс имеет модификатор доступа private и виден только для блока, где он находится. Поэтому нельзя использовать, например, его конструктор в других блоках.

Построим классы Office1 и Human1, которые отличаются от классов Office и Human тем, что класс Human1 вложен в класс Office1. Для класса Office1 мы не можем использовать конструктор без параметров, так как в этом случае в процедуре Main() мы должны будем вызвать конструктор класса Human1, но этот класс недоступен с Main(). Поэтому в классе Office1 обязательно нужно создавать конструктор с параметрами. Получим следующий код для описания класса Office1

```
class Office1
{
    // описание полей класса Office1
    private Human1 boss;
    private string name;
    private int countworker;

    // описание вложенного класса Human1
    private class Human1
    {
        private string name;
        private string secondname;
        private string lastname;
        private int age;
        public string FullName
        {
            get
            {
         return name+ " " + secondname + " " + lastname;
```

```
        }
    }
    public void Info()
    {
        Console.WriteLine(lastname + " " + name + " "
            + secondname + "," + age + Option());
    }

    private string Option()
    {
        string s;
        if (age > 10 && age <= 20)
            s = " лет";
        else
        {
            switch (age % 10)
            {
                case 1:
                    s = " год";
                    break;
                case 2:
                case 3:
                case 4:
                    s = " года";
                    break;
                default:
                    s = " лет";
                    break;
            }
        }
        return s;
    }
    public Human1(string lastname,string name,
       string secondname, int age)
    {
        this.lastname = lastname;
        this.name = name;
        this.secondname = secondname;
        this.age = age;
    }
}
```

```
    //описание метода Info класса Office1
    public void Info()
    {
        Console.WriteLine("Подразделение: {0}, работающих
             {1} человек", name, countworker);
        boss.Info();
    }

    // описание конструктора класса Office1
    public Office1(string lastname, string name,
          string secondname, int age, string name1,
          int countworker)
    {
        this.name = name1;
        this.countworker = countworker;
        boss = new Human1(lastname,name,secondname,age);
    }
}
```

В процедуру `Main()` запишем код

```
Console.WriteLine("\nВложенные классы");
Office1 off3 = new Office1("Попов", "Иван", "Петрович",
              41, "кафедра химии", 9);
off3.Info();
```

В результате получим

```
Вложенные классы
Подразделение: кафедра физики, работающих 15 человек
Алексеев Семен Михайлович, 53 года
```

Вложенный класс имеет смысл использовать тогда, когда его экземпляр используется только в определенном классе. При использовании вложенных классов улучшается читаемость кода, и если нас не интересует устройство основного класса, то разбирать работу вложенного класса нет необходимости.

§ 1.8. Перегрузка операций

Выше мы рассмотрели описание полей, свойств, методов и конструкторов. Можно для экземпляров класса ввести некоторые

операции. Как мы знаем, операции делятся на бинарные и унарные. В бинарных операциях участвуют два операнда. К ним, например, относится сложение, сравнение и т.п. В унарных операциях участвует один операнд, например, взятие противоположного элемента (знак '-'), отрицание и т.п. Для обозначения операций обычно используются соответствующие символы. Можно эти символы использовать для обозначения операций в своем классе. В результате мы получим перегрузку операции на операнды определенного нами типа. Необходимо иметь в виду, что для бинарной операции необходимо использовать символ, предназначенный для бинарных операций, для унарной – для унарных. Описание операции осуществляется в виде функции в теле класса. Заголовок функции записывается так:

```
public static тип operator символ (параметры)
```

Здесь тип – тип возвращаемого значения, символ - символ, который будет использоваться для обозначения операции, параметры – один или два параметра в зависимости от того, какая операция описывается, унарная или бинарная. В теле метода описывается реализация операции.

В качестве примера приведем описание двух операций. Первая операция «увеличение возраста» будет увеличивать на год возраст сотрудника. Очевидно, что для выполнения этой операции необходим соответствующий экземпляр класса Human. Следовательно, эта операция унарная. Для нее будем использовать символ «!». Вторая операция «изменение фамилии» предназначена для изменения фамилии сотрудника. В ней участвуют два операнда: первый имеет тип Human, второй – string для новой фамилии. Поэтому эта операция бинарная. Для второй операции будем использовать символ «/». В описание класса Human добавим фрагмент

```
public static Human operator ! (Human a)
{
     return new Human(a.lastname, a.name, a.secondname,
       a.age + 1);
}
public static Human operator / (Human a, string s)
{
     return new Human(s, a.name, a.secondname, a.age);
}
```

Здесь мы в первой операции по вводимому операнду `a` создаем и возвращаем новый экземпляр, у которого значение поля `age` на единицу больше. Во второй операции создаем и возвращаем новый экземпляр с измененным значением поля `lastname`. В процедуру `Main()` запишем код

```
Console.WriteLine("\nОписание операций");
Human hmn4 = new Human("Васильева", "Александра",
         "Павловна", 24);
hmn4.Info();
hmn4 = !hmn4;
hmn4.Info();
hmn4 = hmn4/"Петрова";
hmn4.Info();
```

После его выполнения получим

```
Васильева Александра Павловна, 24 года
Васильева Александра Павловна, 25 лет
Петрова Александра Павловна, 25 лет
```

Мы видим, что после применения первой операции возраст на год увеличился, а после применения второй – изменилась фамилия.

§ 1.9. Массивы пользовательских объектов

На основе созданного класса можно определить массив элементов этого типа. Для этого достаточно создать требуемые экземпляры данного класса, описать массив обычным образом и эти экземпляры указать в качестве значений. Для полученного массива можно будет применять соответствующие свойства и методы класса `System.Array`.

Рассмотрим пример. Создадим массив соответствующий классу `Human` и найдем средний возраст сотрудников. Для этого в процедуру `Main()` запишем код

```
Human hmn1 = new Human("Иванов", "Василий",
      "Петрович", 45);
Human hmn2 = new Human("Титов", "Петр", "Николаевич", 46);
Human hmn3 = new Human("Васильева", "Александра",
       "Павловна", 24);
Human hmn4 = new Human("Алексеев", "Федор",
       "Никифорович", 33);
```

```
Human hmn5 = new Human("Шишкин", "Константин",
       "Алексеевич", 37);
Human hmn6 = new Human("Николаев", "Алексей",
       "Иванович", 41);
Human[] a = { hmn1, hmn2, hmn3, hmn4, hmn5, hmn6 };
double s = 0;
for (int k = 0; k < a.Length; k++)
     s += a[k].Age;
s = Math.Round(s / a.Length, 1);
Console.WriteLine("Средний возраст сотрудников - {0}", s);
```

После выполнения получим

```
Вы используете класс Human
Средний возраст сотрудников – 37,7
```

Здесь мы воспользовались свойством `Length` класса `System.Array`.

§ 1.10. Структуры

Структуры можно рассматривать как упрощенная разновидность классов. Рассмотрим принципы создания структур. Они создаются по аналогии с обычными классами, только вместо ключевого слова `class` используется слово `struct`. В теле структуры, как обычно, описываются поля, свойства, методы и другие члены структуры.

Допускается также определение конструкторов с параметрами. Конструктор без параметров (по умолчанию) доступен всегда. Переопределять его нельзя. Он инициализирует поля значениями, заданными по умолчанию. Затем их значения можно изменять вручную. В теле структуры можно инициализировать только статические поля.

Экземпляры структур можно создать, как обычно, с помощью ключевого слова `new`. Это целесообразно при использовании конструктора с параметрами, инициализирующего поля экземпляра. Можно создать экземпляр, инициализируя поля с помощью свойств. В этом случае будет автоматически создан экземпляр с помощью конструктора по умолчанию.

Принципиальное отличие обычного класса от структуры состоит в том, что класс относится к ссылочному типу данных, а структура – к значимому типу. Ссылочный тип означает, что если мы определим переменную данного типа и создадим соответствующий объект, то в ячейку переменной запишется адрес объекта. Если создать вторую переменную этого же типа и ей присвоить значение первой, то получим,

что в двух переменных записаны ссылки на один и тот же объект. Поэтому при изменении полей первой переменной будут изменяться поля и у второй переменной. В случае значимого типа ситуация другая. Здесь переменная соответствует объекту. Если присвоим второй переменной значение первой, то будет создана копия объекта, и она будет поставлена в соответствие второй переменной, т.е. будут два объекта. При изменении полей первой переменной поля второй меняться не будут. Покажем это.

Создадим класс `HumanCl` и структуру `HumanSt` практически с одинаковыми членами. Поля, свойства и методы и них совпадают и определяются так:

```
private string name;
private string secondname;
private string lastname;
private int age;
public string Name
{
    set
    {
        name = value;
    }
}
public string SecondName
{
   set
   {
       secondname = value;
   }
}
public string LastName
{
  set
  {
      lastname = value;
  }
}
public int Age
{
  set
  {
     age = value;
  }
```

```
}
public void Info()
{
    Console.WriteLine(lastname + " " + name + " " +
      secondname + "," + age + s);
}

private string Option()
{
    string s;
    if(age > 10 && age <= 20)
        s = " лет";
    else
    {
        switch(age%10)
        {
            case 1: s = " год";
               break;
            case 2:
            case 3:
            case 4: s = " года";
               break;
            default: s = " лет";
               break;
        }
    }
    return s
     }
```

Отличие состоит только в конструкторах. В структуре `HumanSt` опишем конструктор с параметрами

```
public HumanSt(string lastname, string name, string
      secondname, int age)
{
    this.lastname = lastname;
    this.name = name;
    this.secondname = secondname;
    this.age = age;
}
```

В классе HumanCl введем два конструктора. Один конструктор без параметров (в структуре он определен автоматически):

```
public HumanCl()
{
}
```

а второй - с параметрами по аналогии с конструктором структуры.

Запишем следующий код в процедуру Main():

```
Console.WriteLine("используем класс");
Human1 h1=new HumanCl("Иванов","Василий","Петрович",46);
Human1 h2 = new HumanCl();
h2 = h1;
Console.WriteLine("до изменения");
h1.Info();
h2.Info();
Console.WriteLine("после изменения");
h1.Name = "Семен";
h1.LastName = "Сидоров";
h1.SecondName = "Алексеевич";
h1.Age = 42;
h1.Info();
h2.Info();

Console.WriteLine("используем структуру");
Human2 h3=new HumanSt("Иванов","Василий","Петрович",46);
Human2 h4 = wrk3;
Console.WriteLine("до изменения");
h3.Info();
h4.Info();
Console.WriteLine("после изменения");
h3.Name = "Семен";
h3.LastName = "Сидоров";
h3.SecondName = "Алексеевич";
h3.Age = 42;
h3.Info();
h4.Info();
```

После выполнения получим

```
используем класс
до изменения
Иванов Василий Петрович, 46 лет
Иванов Василий Петрович, 46 лет
после изменения
Сидоров Семен Алексеевич, 42 года
Сидоров Семен Алексеевич, 42 года
используем структуру
до изменения
Иванов Василий Петрович, 46 лет
Иванов Василий Петрович, 46 лет
после изменения
Сидоров Семен Алексеевич, 42 года
Иванов Василий Петрович, 46 лет
```

Разберем результат. Вначале мы создали два экземпляра класса, обозначенные двумя переменными. Поля первого объекта заполнили при создании конструктором. Затем второй переменной присвоили первую. Поля обеих переменных вывели на консоль. Получили одинаковые строки.

После этого изменили поля первой переменной и, опять, поля обеих переменных вывели на экран. Получили одинаковые, но измененные строки. Это означает, что мы имеем дело с одним объектом, просто под двумя именами.

Далее, мы аналогичные действия проделали с двумя экземплярами структуры. Как видим, в этом случае изменения отразились только на полях первой переменной, т.е. мы имеем дело с двумя объектами. Когда второй переменной присваивали первую, была создана копия объекта, и вторая переменная означает именно копию. Оригинал мы изменили, а копия сохранилась.

Отметим, что использование данных значимого типа ускоряет выполнение программы, так как экономятся вычислительные ресурсы. К данным ссылочного типа обращение идет через ссылку, а к значимым данным – напрямую. Поэтому структуры подходят для создания несложных типов, не требующих использования наследования. Примерами структур в стандартной библиотеке классов `.Net` являются такие типы, как `int, float, double, bool` и многие другие. Вообще значимые типы описываются с помощью структур, поэтому их также называют структурными типами.

§ 1.11. Частичные классы и методы

Выше мы говорили, что при добавлении класса создается файл с указанным именем, и в нем располагается шаблон создаваемого класса. При необходимости можно получить доступ к этому файлу через `Обозреватель решения`. В данном случае мы имеем, что каждый класс располагается в отдельном файле и представляет собой программный модуль, который можно использовать и в других проектах. Если же в проекте используется достаточно большое количество относительно простых классов, например, для учебных целей, то можно, в принципе, описания классов поместить в одном файле друг под другом.

Возможна и противоположная ситуация, когда описание одного класса располагается в нескольких файлах, т.е. мы можем иметь несколько файлов с определением одного и того же класса, и при компиляции все эти определения будут скомпилированы в одно. Соответствующие фрагменты описания класса называются частичными классами. У них в заголовке добавляется слово `partial`. Распределение членов класса по файлам произвольно. Это не влияет на окончательный результат. Распределение обычно выполняется с учетом некоторых поставленных целей. Примером может служить описание класса `Form` в Windows-приложениях. Один файл генерируется автоматически при создании пользовательского интерфейса, а во втором файле располагаются событийные процедуры, разрабатываемые пользователями.

В качестве примера опишем класс `Student`. Создадим в проекте два файла с кодом. Не столь важно как эти файлы будут называться, например, `Student1.cs` и `Student2.cs`. В одном из этих файлов (без разницы в каком именно) определим класс:

```
partial class Student
{
    private string name;
    public void G() => Console.WriteLine(grup);
}
```

В другом файле определим класс

```
partial class Student
{
    private string grup;
    public Student(string name,string grup)
    {
        this.name = name;
```

```
        this.grup = grup;
    }
    public void F() => Console.WriteLine(name);
}
```

Таким образом, два файла в проекте содержат описание одного класса Student, который включает в себя определение двух полей, двух методов и конструктора с параметрами. Оба определенных здесь класса являются частичными.

Мы можем использовать все методы класса Student. Для этого запишем в процедуру Main() код

```
Console.WriteLine("Частичные классы и методы");
Student st = new Student("Иванов", "9ИС01");
st.F();
st.G();
```

при выполнении которого получим

```
Частичные классы и методы
Иванов
9ИС01
```

Частичные классы могут содержать частичные методы. В заголовке таких методов также указывается ключевое слово partial. Причем определение частичного метода без тела метода находится в одном частичном классе, а реализация этого же метода - в другом частичном классе.

Например, изменим выше определенные классы Student следующим образом

```
partial class Student
{
    private string name;
    public void G()
    {
        Console.WriteLine(grup);
    }
    partial void F1();
    public void G1()
    {
```

```
        F1();
    }
}
```

и

```
partial class Student
{
    private string grup;
    public Student(string name,string grup)
    {
        this.name = name;
        this.grup = grup;
    }
    public void F()
    {
        Console.WriteLine(name);
    }
    partial void F1()
    {
        Console.WriteLine(name + "  " + grup);
    }
}
```

В первом частичном классе определен новый метод `F1()`. Причем на момент определения этого класса неизвестно, что представляет собой этот метод, какие действия он будет выполнять. Тем не менее, мы знаем список его параметров и можем его здесь вызвать в теле другого метода `G1()`. Само же тело этого метода определяется в другом частичном классе.

Добавим в процедуру `Main()` оператор

```
st.G1();
```

и дополнительно получим строку

```
Иванов  9ИС01
```

Стоит отметить, что по умолчанию к частичным методам применяется ряд ограничений:

- они не могут иметь модификаторы доступа;
- они имеют тип `void`;
- они не могут иметь выходные параметры;

- они не могут иметь модификаторы `virtual`, `override`, `sealed`, `new` или `extern`.

§ 1.12. Анонимные типы

Иногда возникает необходимость использования отдельного объекта, обладающего определенным набором полей с некоторыми фиксированными значениями. Более того, если для этого объекта понадобится добавление новых полей, то это приведет к необходимости изменения соответствующего класса. Поэтому в таких ситуациях желательно иметь инструмент создания объекта без описания класса. Это позволяют анонимные типы. Их суть состоит в возможности непосредственного создания объекта с определенным набором полей без описания класса. Для этого используется оператор вида

```
var переменная = new {поле1 = значение1,
         поле2 = значение2, поле3 = значение3,...};
```

После создания объект можно использовать обычным образом. При этом необходимо иметь в виду, что поля доступны только для чтения, и их значения невозможно изменить, а можно только переопределить.

Рассмотрим пример. Запишем в процедуру `Main()` операторы

```
var user = new {name = "Алексеев", group = "1ИС03"};
Console.WriteLine("Имя пользователя {0}, группа {1}",
       user.name, user.group);
user = new {name = "Алексеев", group = "1ИС04"};
Console.WriteLine("Имя пользователя {0}, группа {1}",
       user.name, user.group);
```

При их выполнении получим

```
Имя пользователя Алексеев, группа 1ИС03
Имя пользователя Алексеев, группа 1ИС04
```

Заметим, что для изменения группы нам пришлось объект переопределить. Мы не могли просто записать

```
user.group = "1ИС04";
```

Это привело бы к ошибке.

Хотя мы не описывали класс, а непосредственно создали объект, при компиляции для этого объекта все равно будет автоматически создаваться соответствующий класс. Типы, описываемые такими классами, называются анонимные. Все они относятся к ссылочным типам.

Эти классы имеют имена, которые начинаются с выражения

```
<>f__AnonymousType
```

дальше имя включает в себя количество полей и их типы.

Если в программе используются несколько объектов анонимных типов с одинаковым набором полей, то для них компилятор создаст одно определение анонимного типа.

Глава 2. Дочерние классы

Классы обладают важным свойством: на основе одного класса, который называется родительским, или базовым, можно создавать другие классы, которые называются дочерними, или производными. Этот процесс можно продолжить. Все классы, созданные на основе некоторого класса, называются его потомками.

Заметим, что для структур такая возможность отсутствует. Все структуры создаются на основе базового класса `System.ValueType` и нет возможности на основе одной структуры создавать другую.

Если мы хотим запретить создание дочерних классов. Для этого при описании класса используется модификатор `sealed`, например,

```
sealed class MyClass
```

Теперь попытки создания дочернего класса для `MyClass` приведут к ошибке.

§ 2.1. Наследование

Дочерние классы обладают всеми свойствами и методами родительского класса. Этот факт называется наследованием. Кроме того, можно задавать новые свойства и методы.

Для класса `Human` создадим дочерний класс `Worker`. В этот класс добавим поле `tarif`, которое означает тарифную сетку (в данном случае оплату за один рабочий день), и метод `ZarPlata`, который вычисляет заработную плату работника, как произведение `tarif` на количество отработанных дней. Количество отработанных дней (`countday`) будем вводить в качестве параметра метода. Устанавливать значение `tarif` будем с помощью свойства `Tarif`. Получим следующий код:

```
class Worker:Human
{
    private int tarif;
    public int Tarif
    {
        set
        {
            tarif = value;
        }
    }
    public int ZarPlata(int countday)
```

```
    {
        return tarif * countday;
    }
}
```

При создании класса мы записали

```
class Worker:Human
```

Это означает, что класс `Worker` является дочерним для класса `Human`.

В процедуру `Main()` запишем следующий код:

```
Worker wrk1 = new Worker();
wrk1.LastName = "Сидоров";
wrk1.Name = "Александр";
wrk1.SecondName = "Михайлович";
wrk1.Age = 42;
wrk1.Tarif = 2000;
wrk1.Info();
Console.WriteLine("Сотрудник {0}, зарплата {1}",
       wrk1.FullName, wrk1.ZarPlata(20));
```

После выполнения кода получим

```
Вы используете класс Human
Сидоров Александр Михайлович, 42 года
Введите пароль
456
Сотрудник Сидоров Александр Михайлович, зарплата 40000
```

Мы создали объект с помощью конструктора без параметров класса `Worker` и воспользовались методами и свойствами этого класса. Как видим, все свойства и методы родительского класса `Human` также доступны в новом классе. Как обычно, при использовании свойства `FullName` было необходимо ввести пароль.

Заметим, что первая выведенная строка означает, что мы используем класс `Human`. Это сработал статический конструктор данного класса.

§ 2.2. Полиморфизм

При создании дочернего класса можно изменять реализацию некоторых методов. Это называется полиморфизмом. Таким образом, реализации методов в родительском классе и дочерних классах могут различаться.

Наличие полиморфизма обеспечивает гибкость при создании дочерних классов. Мы не можем отменить методы родительского класса. Зато есть возможность их переопределить. Это бывает актуально, когда для экземпляров дочернего класса существуют более простые алгоритмы реализации методов, чем для экземпляров родительского класса. Например, пусть родительский класс – это треугольники, метод позволяет найти периметр треугольника, а дочерний класс – это равносторонние треугольники. Естественно их периметры определяются по более простой формуле, и нет необходимости использовать общую формулу.

Переопределим в классе `Worker` метод `Info(),` чтобы он дополнительно выводил значение поля `tarif.` Для этого в код класса `Worker` добавим фрагмент описания метода

```
public new void Info()
{
     Console.WriteLine(lastname + " " + name + " " +
       secondname + "," + age + Option() + ", " + tarif
       + " руб/день");
}
```

Как видим, при записи заголовка метода мы добавили ключевое слово `new`, которое в данном случае означает изменение реализации существующего метода. Мы видим, что здесь используются поля `lastname, name, secondname, age` и метод `Option()` класса `Human`. Все они имеют модификаторы доступа `private,` поэтому доступны только внутри класса `Human`. Чтобы обеспечить доступ к указанным членам класса `Human` методов классов-потомков, в частности класса `Worker`, следует заменить модификаторы доступа `private` модификаторами `protected`.

Содержимое процедуры `Main()` оставим без изменения, но после выполнения кода теперь получим строки

```
Вы используете класс Human
Сидоров Александр Михайлович, 42 года, 2000 руб/день
Введите пароль
456
Сотрудник Сидоров Александр Михайлович, зарплата 40000
```

Как видим, при выводе второй строки использовалась реализация метода `Info()`, описанная в дочернем классе.

§ 2.3. Конструктор дочернего класса

Выше, при создании экземпляра дочернего класса мы использовали его конструктор по умолчанию. Создадим конструктор с параметрами

```
public Worker(string lastname, string name,
     string secondname, int age, int tarif)
{
    this.lastname = lastname;
    this.name = name;
    this.secondname = secondname;
    this.age = age;
    this.tarif = tarif;
}
```

который будет инициализировать все поля при создании экземпляра класса `Worker.`

Учитывая, что при определении конструктора с параметрами у класса `Worker` конструктор без параметров этого класса автоматически не создается, опишем его явно в виде

```
public Worker()
{
}
```

Добавим этот конструктор в класс `Worker` и вернемся к конструктору с параметрами. Конструкторы дочерних классов можно создавать на основе конструкторов родительских классов. Например, рассматриваемый конструктор можно описать так:

```
public Worker(string lastname, string name,
      string secondname, int age, int tarif) :
      base(lastname, name, secondname, age)
{
    this.tarif = tarif;
}
```

Здесь строка в заголовке метода

```
: base(lastname, name, secondname, age)
```

означает, что используется конструктор базового класса с указанными в скобках параметрами. В теле создаваемого конструктора будет предполагаться присутствие операторов тела указанного конструктора базового класса и будет необходимо записать только недостающие операторы. В данном случае это оператор

```
this.tarif = tarif;
```

Так можно сократить запись. Приведенный код добавим в класс `Worker.` Процедуру `Main`() перепишем следующим образом:

```
Worker wrk1 = new Worker();
wrk1.LastName = "Сидоров";
wrk1.Name = "Александр";
wrk1.SecondName = "Михайлович";
wrk1.Age = 42;
wrk1.Tarif = 2000;
Console.WriteLine("Сотрудник {0}, зарплата {1}",
     wrk1.FullName, wrk1.ZarPlata(20));
Worker wrk2 = new Worker("Иванов", "Василий", "Петрович",
     45, 1500);
wrk2.Info();
```

Здесь мы объект `wrk1` создаем с помощью конструктора без параметров, а объект `wrk2` – с помощью конструктора с параметрами. После выполнения получим

```
Вы используете класс Human
Введите пароль
456
Сотрудник Сидоров Александр Михайлович, зарплата 40000
Иванов Василий Петрович, 45 лет, 1500 руб/день
```

§ 2.4. Запись экземпляра дочернего класса в экземпляр родительского класса

Рассмотрим соотношения между экземплярами родительского и дочернего классов. Дочерний класс получается из родительского класса добавлением новых свойств и методов. Поэтому дочерний класс более узкий и его экземпляры в принципе, входят и в родительский класс. С

точки зрения преобразования типов, это означает, что существует неявное преобразование дочернего типа в родительский тип, т.е. допустимы операторы

```
Worker wrk =
     new Worker("Иванов","Василий","Петрович",45,1500);
Human wrk1 = wrk;
```

или

```
Human wrk1 =
      new Worker("Иванов","Василий","Петрович",45,1500);
```

В первом случае мы вначале создали экземпляр `wrk` класса `Worker`, а затем его записали в экземпляр `wrk1` класса `Human`. Во втором же случае сразу созданный экземпляр класса `Worker` записали в экземпляр `wrk1` класса `Human`.

Заметим, что хотя `wrk1` фактически указывает на экземпляр класса `Worker,` этому экземпляру доступны только свойства и методы класса `Human.` Например, мы не можем для этого экземпляра использовать свойство `Tarif`, метод `ZarPlata()`, а метод `Info()` будет выполняться на основе реализации в классе `Human`.

Как мы знаем, преобразование, обратное неявному преобразованию, всегда должно быть явным. Поэтому преобразование из родительского типа в дочерний тип должно быть явным. Для этого можно использовать операцию приведения типа, например,

```
Worker wrk2 = (Worker) wrk1;
```

Теперь для объекта `wrk2` доступны все свойства и методы класса `Worker`.

При выполнении код

```
Worker wrk1 = new Worker("Иванов", "Василий", "Петрович",
       45, 1500);
wrk1.Info();
Human wrk2 = wrk1;
wrk2.Info();
Worker wrk3 = (Worker)wrk2;
wrk3.Info();
```

получим строки

```
Вы используете класс Human
Иванов Василий Петрович, 45 лет, 1500 руб/день
Иванов Василий Петрович, 45 лет
Иванов Василий Петрович, 45 лет, 1500 руб/день
```

Как видим, объект `wrk2` описан как экземпляр класса Human. Поэтому при выполнении метода `Info()` отсутствует поле `tarif`.

Заметим, что невозможно экземпляр родительского класса записать в экземпляр дочернего класса. Коды

```
Human wrk4 = new Human();
Worker wrk5 = (Worker) wrk4;
```

или

```
Worker wrk5 = new Human();
```

приведут к ошибке. Изначально объект должен быть создан конструктором дочернего класса.

Укажем возможные пути использования изложенного здесь материала. Предположим, что на фирме имеются разные категории работников: рабочие, инженеры, лаборанты и т.п. На основе базового класса `Worker` можно для каждой категории создать свой дочерний класс, учитывающий особенности данной категории работников. Затем при необходимости, например, для начисления заработной платы, можно создать массив типа `Worker` и записать туда экземпляры дочерних классов. Это позволит одним циклом всем работникам начислить зарплату.

§ 2.5. Виртуальные методы

Как мы видели, переменная родительского класса может использоваться для хранения экземпляра дочернего класса. Однако при этом мы не сможем использовать методы из дочернего класса. Если имеется некоторый метод, переопределенный в дочернем классе, то для такого экземпляра будет использоваться реализация из родительского класса. Это не всегда удобно. Избежать данного факта можно при помощи виртуальных методов.

При описании метода в родительском классе используется ключевое слово `virtual`, а в дочернем классе – ключевое слово `override`. Здесь слово `new` не используется. Теперь для указанных выше экземпляров дочернего класса будет использоваться реализация метода из дочернего класса.

Вернемся к вышерассмотренному примеру. В классы Human и Worker добавим метод Info1(), тело которого совпадает с телом метода Info(), описанного в каждом из этих классов, а заголовки имеют вид

```
virtual public void Info1()
```

в классе Human и

```
override public void Info1()
```

в классе Worker.

В процедуру Main() добавим код

```
Console.WriteLine("\nИспользование виртуального метода!”);
wrk1.Info1();
wrk2.Info1();
wrk3.Info1();
```

Здесь мы для ранее созданных экземпляров используем метод Info1(). После выполнения получим строки

```
Вы используете класс Human
Иванов Василий Петрович, 45 лет, 1500 руб/день
Иванов Василий Петрович, 45 лет
Иванов Василий Петрович, 45 лет, 1500 руб/день

Использование виртуального метода!
Иванов Василий Петрович, 45 лет, 1500 руб/день
Иванов Василий Петрович, 45 лет, 1500 руб/день
Иванов Василий Петрович, 45 лет, 1500 руб/день
```

Как видим, что вне зависимости от типа переменной (Human или Worker), которая использована для записи экземпляра класса Worker, всегда использовалась реализация метода Info1() из класса Worker. Для метода Info() это было не так.

§ 2.6. Класс System.Object

Все классы C#, в том числе и пользовательские, являются потомками класса System.Object. Это происходит неявным образом даже, если никакой класс не указан в качестве родительского класса. Хотя это можно указать и явно, например,

```
class Human: System.Object
```

Хотя результат от этого не изменится.

Из того, что любой класс является потомком класса `System.Object` следует, что можно экземпляр любого класса записать в переменную типа `System.Object` и для объектов любого класса доступны методы класса `System.Object`.

Первый факт означает, что правомерным является, например, оператор

```
Object a = new Human();
```

Приведем методы класса `System.Object`:

- `Equals()` - виртуальный метод, возвращающий `True`, если два объекта расположены в одном месте памяти (второй объект передается в качестве параметра).
- `GetHashCode()` - виртуальный метод, возвращает некоторое целое число (хэш-код), однозначно идентифицирующее экземпляр класса.
- `GetType()` - возвращает объект типа `Type`, описывающий соответствующий тип.
- `ToString()` - виртуальный метод, по умолчанию возвращает строку, представляющую полное имя типа объекта.

При выполнении кода

```
Object ob1, ob2, ob3;
ob1 = new System.Object();
ob2 = ob1;
Console.WriteLine(ob1.Equals(ob2));
ob3 = new System.Object();
Console.WriteLine(ob1.Equals(ob3));
Console.WriteLine(ob1.GetHashCode());
Console.WriteLine(ob2.GetHashCode());
Console.WriteLine(ob3.GetHashCode());
Console.WriteLine(ob1.GetType());
Console.WriteLine(ob1.ToString());
```

получим

```
True
False
49147025
49147025
1762126
System.Object
System.Object
```

Тот факт, что для переменных `ob1` и `ob2` получились одинаковые хэш-коды, означает, что эти переменные указывают на один экземпляр. В то время, как `ob3` указывает на другой экземпляр. Именно об этом говорят первые две строки.

Три метода из четырех указанных являются виртуальными. Поэтому они, как правило, в дочерних классах переопределяются. Рассмотрим экземпляры класса `System.String`. Выполним код

```
Object ob1, ob2, ob3;
ob1 = "Hello";
ob2 = ob1;
Console.WriteLine(ob1.Equals(ob2));
ob3 = "Hello";
Console.WriteLine(ob1.Equals(ob3));
Console.WriteLine(ob1.GetHashCode());
Console.WriteLine(ob2.GetHashCode());
Console.WriteLine(ob3.GetHashCode());
Console.WriteLine(ob1.GetType());
Console.WriteLine(ob1.ToString());
```

Результат примет вид

```
True
True
-694847
-694847
-694847
System.String
Hello
```

Как видим, все три переменные указывают на один объект, и метод `ToString()` выводит саму строку, а не имя типа объекта.

Рассмотрим арифметический тип. При выполнении кода

```
Object ob1, ob2, ob3;
ob1 = 3;
ob2 = ob1;
Console.WriteLine(ob1.Equals(ob2));
ob3 = 3;
Console.WriteLine(ob1.Equals(ob3));
Console.WriteLine(ob1.GetHashCode());
Console.WriteLine(ob2.GetHashCode());
Console.WriteLine(ob3.GetHashCode());
Console.WriteLine(ob1.GetType());
Console.WriteLine(ob1.ToString());
```

получим

```
True
True
3
3
3
System.Int32
3
```

В этом случае хэш-код совпадает со значение переменной и метод `ToString()` также выводит ее значение.

Во всех этих примерах мы использовали встроенные классы. Теперь рассмотрим пример с классом `Human`, введенным ранее. Выполним код

```
Object ob1, ob2, ob3;
ob1 = new Human("Васильева","Александра","Павловна", 24);
ob2 = ob1;
Console.WriteLine(ob1.Equals(ob2));
ob3 = new Human("Васильева","Александра","Павловна", 24);
Console.WriteLine(ob1.Equals(ob3));
Console.WriteLine(ob1.GetHashCode());
Console.WriteLine(ob2.GetHashCode());
Console.WriteLine(ob3.GetHashCode());
Console.WriteLine(ob1.GetType());
Console.WriteLine(ob1.ToString());
```

Получим

```
Вы используете класс Human
```

```
True
False
46104728
46104728
12289376
Project.Human
Project.Human
```

Заметим, хотя значения полей у всех одинаковые, однако объект `ob3` отличается от объектов `ob1` и `ob2`, а метод `ToString()` опять выводит имя типа объекта.

Переопределим методы следующим образом. Пусть метод `Equals()` выводит `True`, если экземпляры имеют одинаковые значения полей, а метод `ToString()` выводит данные экземпляра. В описание класса `Human` добавим методы

```
public override bool Equals(object ob)
{
     Human hmn = (Human)ob;
     return name == hmn.name && lastname == hmn.lastname
&&
        secondname == hmn.secondname && age == hmn.age;
}
public override string ToString()
{
     return name + " " + secondname + " " + lastname +
         ", "+age;
}
```

Теперь после выполнения программы получим

```
Вы используете класс Human
True
True
46104728
46104728
12289376
Project.Human
Александра Павловна Васильева, 24
```

На значение хэш-кода внимание обращать не следует. Мы их выводим только для проверки: у нас один и тот же экземпляр или разные. В данном случае по хэш-кодам мы видим, что экземпляры `ob1` и `ob3` различаются, но метод `Equals()` выводит `True` из-за того, что у них данные совпадают. Изменился и результат выполнения метода `ToString()`.

§ 2.7. Абстрактные классы

Предположим, что мы хотим для плоских геометрических фигур определять периметр и площадь. Однако для определения этих характеристик нет общих формул. Существуют формулы для треугольников, четырехугольников, окружности, сегментов и т.п. Мы не можем построить какую-то произвольную фигуры и вычислить по некоторым общим формулам ее периметр и площадь, а должны иметь фигуру определенного типа и воспользоваться соответствующими формулами.

Рассмотрим это с точки зрения программирования. Мы можем создать класс Фигура и указать, что у него есть методы Периметр и Площадь, но не можем описать их реализацию. Такие методы называются абстрактные, и сам класс также называется абстрактным. Абстрактный класс наряду с абстрактными методами может включать и обычные методы с их полной реализацией. Из-за отсутствия реализации некоторых методов нельзя создать экземпляр абстрактного класса. Он может использоваться только как базовый для создания дочерних классов, в которых можно описать реализацию абстрактных методов. Мы можем создать дочерние классы Треугольник, Четырехугольник, Окружность, Сегмент и т.п., в каждом из которых привести реализации методов Периметр и Площадь. Теперь мы можем создавать экземпляры дочерних классов и использовать методы Периметр и Площадь. Если мы опишем не все абстрактные методы, то и дочерний класс окажется абстрактным и нельзя будет создать его экземпляры. Этот класс опять можно будет использовать только для создания дочернего.

Мы могли бы создать обычный класс Фигура и вообще не включать в него методы Периметр и Площадь. Затем создать дочерние классы, в которых определить эти методы. Вроде бы мы также решаем нашу задачу. Разница состоит в том, что в этом варианте в описании родительского класса вообще отсутствуют нужные нам методы, ради которых мы и создаем классы. Это усложняет понимание назначений классов в данной программе.

Таким образом, абстрактный класс служит шаблоном для создания других классов. Мы видим, какие обязательные методы должны в них

присутствовать, хотя в родительском классе пока невозможно их реализовать.

При описании абстрактных методов и самого абстрактного класса используется модификатор `abstract`.

Приведем примеры. Вначале рассмотрим пример с геометрическими фигурами. Определим абстрактный класс `Figure` с двумя абстрактными методами

```
abstract class Figure
{
     public abstract double Square();
     public abstract double Perimeter();
}
```

Создадим классы треугольников и прямоугольников с конкретной реализацией этих методов

```
class Triangle: Figure
{
   double a, b, c; //Стороны
   //Конструктор
   public Triangle(double a, double b, double c)
   {
       this.a = a;
       this.b = b;
       this.c = c;
   }
   public override double Square()
   {
   //Используем формулу Герона
       double p = (a + b + c) / 2;
    return Math.Sqrt(p * (p - a) * (p - b) * (p - c));
   }
   public override double Perimeter()
   {
       return a + b + c;
   }
}

class Rectangle: Figure
{
    double a, b; //Стороны
```

```
    //Конструктор
    public Rectangle(double a, double b)
    {
        this.a = a;
        this.b = b;
    }
    public override double Square()
    {
        return a * b;
    }
    public override double Perimeter()
    {
        return (a + b) * 2;
    }
 }
```

Теперь в процедуру `Main()` запишем код

```
Figure f1, f2;
f1 = new Triangle(3, 4, 5);
f2 = new Rectangle(2, 6);
Console.WriteLine(f1.Perimeter() + ", " + f1.Square());
Console.WriteLine(f2.Perimeter() + ", " + f2.Square());
```

При выполнении получим результат

```
12, 6
16, 12
```

Заметим, что мы не можем создавать экземпляр абстрактного класса, но можем использовать переменные этого типа. Здесь переменные `f1`, `f2` объявлены типа `Figure`.

Рассмотрим другой пример. Выше мы на основе класса `Human` определили класс `Worker`, в который добавили метод для вычисления зарплаты. Причем зарплата определялась через тариф. На фирме могут работать разные категории работников. Каждый из них получает зарплату, а определяться она может разными способами. У кого-то оклад, у кого-то сдельная оплата труда, у кого-то зависит, например, от продаж и т.д. Можно на основе `Human` создать для каждой категории свой класс и в каждом из них определить метод начисления зарплаты.

Создадим абстрактный класс `Human` следующего вида

```
abstract class Human
{
    //поля
    protected string name;
    protected string secondname;
    protected string lastname;
    protected int age;
    //абстрактный метод
    public abstract   int ZarPlata();
    //методы с реализацией
    public void Info()
    {
         Console.WriteLine("{0} {1} {2}, {3} {4},
           зарплата {5} руб", lastname, name,
           secondname, age, Option(), ZarPlata());
    }

    private string Option()
    {
        string s;
        if (age > 10 && age <= 20)
            s = " лет";
        else
        {
            switch (age % 10)
            {
                case 1:
                    s = " год";
                    break;
                case 2:
                case 3:
                case 4:
                    s = " года";
                    break;
                default:
                    s = " лет";
                    break;
            }
        }
        return s;
    }
```

```
    //конструктор
    public Human(string lastname, string name, string
          secondname, int age)
    {
       this.lastname = lastname;
       this.name = name;
       this.secondname = secondname;
       this.age = age;
    }
}
```

Здесь мы ввели три метода: метод `ZarPlata()` - абстрактный и методы `Info(), Option()` – обычные, с полной реализацией.

Заметим, что в тело метода `Info()` входит вызов абстрактного метода `ZarPlata()`.

Мы также создали конструктор. Хотя его нельзя использовать для создания экземпляров этого класса, есть возможность его использовать при определении конструкторов дочерних классов.

Создадим два дочерних класса: `Worker` и `Ingener`. Они отличаются полями и реализацией метода `ZarPlata()`. Первый класс имеет вид

```
class Worker: Human
{
    private int tarif;
    private int countday;
    public override  int ZarPlata()
    {
        return tarif * countday;
    }
    public Worker(string lastname, string name, string
         secondname, int age, int tarif, int countday)
         : base(lastname, name, secondname, age)
   {
        this.tarif = tarif;
        this.countday = countday;
   }
}
```

и второй класс -

```
class Ingener: Human
{
    private int oklad;
    public override  int ZarPlata()
    {
        return oklad;
    }
    public Ingener(string lastname, string name, string
          secondname, int age, int oklad)
          : base(lastname, name, secondname, age)
    {
        this.oklad = oklad;
    }
}
```

Запишем процедуру Main():

```
Human wrk1 = new Worker("Иванов", "Василий", "Петрович",
     45, 1500, 20);
wrk1.Info();
Human wrk2 = new Ingener("Сидоров", "Семен", "Алексеевич",
     42, 35000);
wrk2.Info();
```

После выполнения получим

```
Иванов Василий Петрович, 45 лет, зарплата 30000 руб
Сидоров Семен Алексеевич, 42 года, зарплата 35000 руб
```

§ 2.8. Интерфейсы

Различие двух абстрактных классов, рассмотренных выше, состоит в том, что первый класс содержал только абстрактные методы, а второй имел и методы с их реализацией. Дальнейшим обобщением первого варианта являются интерфейсы. Интерфейсы – это классы специального вида. Они аналогичны абстрактным классам, имеющим только абстрактные члены, но отличаются от них как описанием, так и применением.

Для создания интерфейса нужно в меню выбрать пункты `Проект -> Добавить класс`. В открывшемся окне выбрать шаблон `Интерфейс` и

ввести имя. Обычно имя интерфейса начинается на букву “I” от слова interface. После нажатия кнопки Ok появится шаблон вида

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace project1
{
    interface имя_интерфейса
    {
    }
}
```

в который, как обычно, надо вписать необходимый код.

В отличие от классов в теле интерфейсов не следует указывать модификаторы доступа. Они всегда предполагаются public. Здесь не должно быть описаний полей, конструкторов и т.п. Нельзя использовать модификатор static.

На основе интерфейса можно создать дочерний класс. При этом все методы интерфейса в этом классе должны быть реализованы. Здесь нельзя, как в случае абстрактного класса, реализовать только часть абстрактных методов и получить абстрактный дочерний класс.

Заметим, что на основе интерфейса можно создать и дочернюю структуру. Интерфейс – это единственное, что может наследоваться структурой.

Перейдем к конкретным кодам. Рассмотрим интерфейс

```
public interface IFigura1
{
    double Square();
    double Perimeter();
}
```

На его основе создадим класс Triangle1, аналогичный рассмотренный выше классу Triangle. Получим

```
class Triangle1: IFigura1
{
    double a, b, c; //Стороны
```

```
    //Конструктор
    public Triangle1(double a, double b, double c)
    {
        this.a = a;
        this.b = b;
        this.c = c;
    }
    public  double Square()
    {
       //Используем формулу Герона
        double p = (a + b + c) / 2;
       return Math.Sqrt(p * (p - a) * (p - b) * (p - c));
    }
    public  double Perimeter()
    {
       return a + b + c;
    }
}
```

Как видим, принципиальной разницы нет. Единственное мы в заголовке класса указали имя интерфейса и в заголовках методов опустили модификатор `override`.

Для сокращения записи мы здесь не рассматриваем класс `Rectangle`. Он создается аналогично.

Приведем код `Main()`

```
Triangle1 f1;
f1 = new Triangle1(3, 4, 5);
Console.WriteLine(f1.Perimeter() + ", " + f1.Square());
```

Здесь необходимо обратить внимание на то, что, в отличие от абстрактных классов, мы не можем при объявлении типа переменной `f1` использовать имя родительского класса. Им является интерфейс.

Интерфейсы являются классами. Поэтому они сами могут наследовать интерфейс, т.е. могут быть дочерними для родительских интерфейсов. Пусть имеется родительский интерфейс

```
public interface ISquare2
{
    double Square();
}
```

На его основе опишем интерфейс IFigura2

```
public interface IFigura2: ISquare2
{
    double Perimeter();
}
```

Здесь метод Square() наследуется от ISquare2, а метод Perimeter() добавляется. Теперь интерфейс IFigura2 используем для описания класса Triangle2, у которого заголовок имеет вид

```
class Triangle2: IFigura2
```

а тело совпадает с телом класса Triangle1 с учетом разницы в имени конструктора.

После выполнения кода

```
Triangle2 f2;
f2 = new Triangle2(3, 4, 5);
Console.WriteLine(f2.Perimeter() + ", " + f2.Square());
```

получим тот же самый ответ.

Отличительной особенностью интерфейсов является возможность одновременного наследования нескольких интерфейсов, т.е. класс может являться дочерним сразу нескольким интерфейсам. Для этого в заголовке класса прописываются все родительские интерфейсы через запятую. Более того, класс может наследовать одновременно один класс и несколько интерфейсов. В этом случае в заголовке сначала пишется имя родительского класса, а за ним имена интерфейсов. Во всех этих случаях дочерний класс должен содержать реализации всех абстрактных методов родителей. Здесь могут возникнуть проблемы, если среди этих методов окажутся одноименные с одинаковой сигнатурой, т.е. совпадают имя, количество параметров и их типы.

В предыдущем примере в дополнение к интерфейсу ISquare2 создадим аналогичный интерфейс IPerimeter3 вида

```
public interface IPerimeter3
{
   double Perimeter();
}
```

а заголовок класса Triangle3 запишем так:

```
class Triangle3 : ISquare2, IPerimeter3
```

т.е. класс наследует два интерфейса. При выполнении соответствующего кода снова получим прежний результат.

Использование интерфейсов, с одной стороны, заставляет разработчиков реализовывать все их методы и отпадает опасность их пропуска, с другой стороны, облегчает понимание возможностей данного класса по ознакомлению с описанием этих интерфейсов.

§ 2.9. Использование классов для хранения данных

Часто при инициализации некоторых полей используются значения из определенных наборов. Например, устанавливая оклад новому работнику, часто работодатель определяет его размер не произвольным образом, а из некоторой таблицы окладов. Или, устанавливая цвет определенного объекта, исходят из соответствующей палитры. Для хранения этих наборов данных могут использоваться классы. Рассмотрим некоторые из таких классов.

Статические члены доступны без создания экземпляров этого класса. Данный факт позволяет использовать классы для хранения некоторых данных и функций с последующим их применением в методах других классов. Такими классами, например, являются класс `Math`, содержащий математические функции и константы, и класс `Convert`, содержащий методы для преобразования типов.

Предположим, что у нас имеется набор данных, которые мы хотим использовать в разных классах. Можно создать специальный класс со статическими полями, в эти поля записать наши данные и создать статические свойства для вывода значений полей. Это позволит их использовать в других кодах. Экземпляры данного класса нас не интересуют. Поэтому можно у конструктора указать модификатор `private`. Мы получим закрытый конструктор и, соответственно, класс без экземпляров. В таких классах можно вводить и статические методы. Они также будут доступны извне.

Рассмотрим пример. Выше использованный класс `Ingener` имеет поле `oklad`. Обычно во многих организациях размер оклада конкретного работника определяется на основе действующей таблицы должностных окладов. Время от времени производится индексирование этих окладов, т.е. увеличение на некоторый процент, например, с учетом инфляции. Создадим класс `Oklad` для хранения этих значений, в который также

включим метод для индексирования оклада. Суть метода будет умножение значения оклада на некоторый повышающий коэффициент.

Приведем описание класса

```
class Oklad
{
    static private int oklad1 = 25000;
    static private int oklad2 = 30000;
    static private int oklad3 = 35000;
    static public int Oklad1
    {
        get
        {
            return oklad1;
        }
    }
    static public int Oklad2
    {
        get
        {
            return oklad2;
        }
    }
    static public int Oklad3
    {
        get
        {
            return oklad3;
        }
    }
    static public void Koeff(float k)
    {
        oklad1 = (int)(oklad1 * k);
        oklad2 = (int)(oklad2 * k);
        oklad3 = (int)(oklad3 * k);
    }
    private Oklad()
    {
    }
}
```

Мы определили три статических поля для хранения значений оклада, описали статические свойства для их вывода и статический метод, который индексирует оклады на свой аргумент. Конструктор без параметров объявили закрытым, что делает невозможным создание экземпляров.

В процедуру `Main()` запишем код:

```
Console.WriteLine("Классы без экземпляров");
Human wrk3 = new Ingener("Иванов", "Василий", "Петрович",
       46, Oklad.Oklad3);
wrk3.Info();
Human wrk4 = new Ingener("Сидоров", "Семен", "Алексеевич",
       42, Oklad.Oklad3);
wrk4.Info();
Oklad.Koeff(1.3f);
Human wrk5 = new Ingener("Иванов", "Василий", "Петрович",
       46, Oklad.Oklad2);
wrk5.Info();
Human wrk6 = new Ingener("Сидоров", "Семен", "Алексеевич",
       42, Oklad.Oklad2);
wrk6.Info();
```

Здесь при создании экземпляров класса Ingener в качестве параметра мы использовали свойства `Oklad2` и `Oklad3` класса `Oklad`, которые определяют размер оклада.

После выполнения кода получим

```
Классы без экземпляров
Иванов Василий Петрович, 46 лет, зарплата 35000 руб
Сидоров Семен Алексеевич, 42 года, зарплата 30000 руб
Иванов Василий Петрович, 46 лет, зарплата 45499 руб
Сидоров Семен Алексеевич, 42 года, зарплата 38999 руб
```

Первые две строки соответствуют результату до индексирования, а две последние строки – после индексирования.

Выше мы для хранения данных использовали статические поля класса без экземпляров. Нам эти поля пришлось объявить закрытыми и создать свойства для чтения. Использование модификатора `public` позволило бы менять значения полей из других методов, что может быть недопустимо. Чтобы это исключить, можно поля объявить константами с помощью ключевого слова `const`. Тогда изменить их уже будет невозможно.

Итак, можно создать абстрактный класс и поля объявить константами. В этом случае невозможно будет создать его экземпляры, и можно будет только считывать значения его полей. В предыдущий проект добавим абстрактный класс `AbsOklad`:

```
abstract class AbsOklad
{
    public const int oklad1 = 25000;
    public const int oklad2 = 30000;
    public const int oklad3 = 35000;
}
```

и в процедуру `Main()` запишем код

```
Console.WriteLine("Константы");
Human wrk7 = new Ingener("Иванов", "Василий", "Петрович",
     46, AbsOklad.oklad3);
wrk7.Info();
Human wrk8 = new Ingener("Сидоров", "Семен", "Алексеевич",
     42, AbsOklad.oklad2);
wrk8.Info();
```

Получим

```
Константы
Иванов Василий Петрович, 46 лет, зарплата 35000 руб
Сидоров Семен Алексеевич, 42 года, зарплата 30000 руб
```

Как видим, класс для хранения значений окладов получился существенно проще, но у нас нет возможности индексировать оклады, так как они объявлены константами.

Для хранения целочисленных констант также можно использовать перечисления. Перечисления представляют собой набор именованных целочисленных констант. К константам можно обращаться по имени. Перечисления описываются так:

```
enum имя_перечисления
{
     константа1,
     константа2,
       ……
}
```

Здесь `константа` – это имя константы возможно с присвоенным ему значением. Константы предполагаются типа `int.` Можно использовать любой другой целочисленный тип, в этом случае он указывается в заголовке через двоеточие.

Константы принимают присваиваемые им значения. Если для некоторой константы инициализация не проводится, то `константа1` принимает значение нуль, а любая другая - на единицу больше, чем предыдущая. Например, в перечислении

```
enum sss
{
      a,
      b,
      c = 5,
      d
}
```

получим `a = 0, b = 1, c = 5, d = 6.`

Перейдем к примеру. Создадим следующее перечисление

```
enum OkladEnum
{
      oklad1 = 25000,
      oklad2 = 30000,
      oklad3 = 35000
}
```

Учтем, что, хотя при инициализации членов перечисления используются значения типа `int`, для обратного перевода значений членов в числовой тип надо использовать явное преобразование. Поэтому код в процедуре в `Main()` можно записать так:

```
Console.WriteLine("Перечисления");
Human wrk9 = new Ingener("Иванов", "Василий", "Петрович",
      46, (int)OkladEnum.oklad3);
wrk9.Info();
Human wrk10 = new Ingener("Сидоров", "Семен",
      "Алексеевич", 42, (int)OkladEnum.oklad2);
wrk10.Info();
```

После выполнения, как и выше, получим

```
Перечисления
Иванов Василий Петрович, 46 лет, зарплата 35000 руб
Сидоров Семен Алексеевич, 42 года, зарплата 30000 руб
```

В отличие от обычных классов перечисления относятся к значимым типам. Поэтому при использовании перечислений программа будет занимать меньше места в памяти и быстрее выполняться.

Все перечисления являются потомками базового класса `System.Enum`, который обладает рядом статических методов. Эти методы можно использовать для работы с перечислениями. Рассмотрим некоторые из них.

Метод `Format` позволяет найти член перечисления по его значению. Этот метод имеет три параметра: имя перечисления, значение искомого члена и режим вывода. Если требуется вывести имя члена с заданным значением, то следует указать режим `"G"`. Еще возможны режимы `"X"` и `"d".` В этих случаях будут выведены значения, соответственно, в шестнадцатеричной и десятичной системах исчисления. Например, операторы

```
Console.WriteLine(Enum.Format(typeof(Stafka),30000,"G"));
Console.WriteLine(Enum.Format(typeof(Stafka),30000,"X"));
Console.WriteLine(Enum.Format(typeof(Stafka),30000,"d"));
```

выведут на консоль строки

```
oklad2
00007530
30000
```

Здесь метод `typeof()` используется для получения объекта `System.Type` для указанного типа.

Рассмотрим второй метод. Метод `GetValues()` позволяет создать массив на основе перечисления. Этот массив будет принадлежать классу `Array`. Чтобы можно было обращаться к его элементам, этот массив следует привести к типу `int[]`, например, с помощью операции приведения к типу. При выполнении операторов

```
int[] a =(int[]) Enum.GetValues(typeof(Stafka));
Console.WriteLine(a[1]);
```

получим

```
30000
```

Глава 3. Обобщения

§ 3.1. Поля для хранения данных неизвестного типа

При создании классов мы описываем поля с указанием типа каждого из них. Именно значения этих типов можно использовать при инициализации данных полей. Однако иногда бывает заранее неизвестно, какие будут использоваться значения для полей. Например, мы хотим создать класс для хранения фамилии сотрудника и некоторой дополнительной информации. Очевидно, что для хранения фамилии следует использовать тип `string`. В то время как тип информации может быть различным и отличаться для разных сотрудников. Возникает вопрос, какой тип использовать при описании этого поля? И на первый взгляд, чтобы выйти из подобной ситуации, мы можем определить поле для информации типа `object`. Так как тип `object` является универсальным типом, от которого наследуется все типы, соответственно в полях подобного типа мы можем сохранить и строки, и числа, и символы, и любые другие объекты. Опишем соответствующий класс:

```
namespace Project1
{
    class Person1
    {
        private string name;
        private object info;
        public string Name
        {
            set
            {
                name = value;
            }
            get
            {
                return name;
            }
        }
        public object Info
        {
            set
            {
                info = value;
```

```
            }
            get
            {
                return info;
            }
        }
        public Person1(string name, object info)
        {
            this.name = name;
            this.info = info;
        }
    }
}
```

Теперь этот класс можно использовать в программе. Если выполнить код:

```
static void Main(string[] args)
{
    Console.WriteLine("Использование типа object");
    Person1 pr1 = new Person1("Иванов", 123);
    Person1 pr2 = new Person1("Петров", "456");
    Person1 pr3 = new Person1("Сидоров", 'f');
    Person1 pr4 = new Person1("Титов", false);
    Person1 pr5 = new Person1("Алексеев", pr1);
    Console.WriteLine("Фамилия {0}, информация {1}",
         pr1.Name, pr1.Info);
    Console.WriteLine("Фамилия {0}, информация {1}",
         pr2.Name, pr2.Info);
    Console.WriteLine("Фамилия {0}, информация {1}",
         pr3.Name, pr3.Info);
    Console.WriteLine("Фамилия {0}, информация {1}",
         pr4.Name, pr4.Info);
    Console.WriteLine("Фамилия {0}, информация {1}",
         pr5.Name, pr5.Info);
}
```

получим

```
Использование типа object
Фамилия Иванов, информация 123
```

```
Фамилия Петров, информация 456
Фамилия Сидоров, информация f
Фамилия Титов, информация False
Фамилия Алексеев, информация Project1.Person1
```

Как видим, нам удается инициализировать поле `info` и вывести его значение, причем для последнего экземпляра мы смогли при инициализации использовать для этого поля значение типа `Person1` и получили название класса. Вид полученного результата связан с тем, что параметр метода `Console.WriteLine()` должен иметь тип `string`, в противном случае к нему применяется метод `ToString()`, который доступен во всех классах.

Хранимые данные, как правило, подвергаются определенной обработке. Для этого могут использоваться операции, соответствующие типу этих данных. В рассматриваемом случае данные разных типов упакованы в объекты типа `object`. Чтобы были доступны соответствующие операции, необходимо выполнить распаковку, например, используя операцию приведения к типу. Для простоты в качестве обработки данных мы будем здесь и далее использовать операции сложения для числовых типов и сцепления для строкового типа. В обоих случаях для обозначения операции применяется символ “+”. Для логического поля используем операцию отрицания. Объектов типа `object` будем применять метод `ToString()`. По полученным результатам мы можем делать выводы о конкретных типах данных. В процедуру `Main()` добавим операторы

```
pr1.Info = (int)pr1.Info + 1;
pr2.Info = (string)pr2.Info + 1;
pr3.Info = (char)pr3.Info + 1;
pr4.Info = !((bool)pr4.Info);
pr5.Info = pr5.Info.ToString();

Console.WriteLine("Фамилия {0}, информация {1}", pr1.Name,
      pr1.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr2.Name,
      pr2.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr3.Name,
      pr3.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr4.Name,
      pr4.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr5.Name,
```

```
pr5.Info);
```

Их выполнение даст результат

```
Фамилия Иванов, информация 124
Фамилия Петров, информация 4561
Фамилия Сидоров, информация 103
Фамилия Титов, информация True
Фамилия Алексеев, информация Project1.Person1
```

Отсюда следует, что для экземпляров `pr1`, `pr2` и `pr4` есть возможность при помощи операций приведения к типу привести значения их полей к типу данных, использованному в конструкторе, и выполнить операции, доступные в соответствующих классах. Для экземпляра же `pr3` получили несколько неожиданный результат. Поступая совершенно аналогично экземплярам `pr1` и `pr2`, здесь мы вместо символа получили число. Это связано с тем, что существует неявное преобразование типа `char` в целочисленные типы, в то время как обратное преобразование всегда должно быть явным. Для экземпляра `pr5` мы только при помощи метода `ToString()` преобразовали в тип `string`, но это не привело к изменению результата по выше указанной причине.

Использование типа `object` для описания полей в подобных ситуациях не является оптимальным. Дело в том, что, как мы уже говорили, в данном случае мы сталкиваемся с такими явлениями как упаковка и распаковка. Так, при передаче в конструктор, например, значения типа `int` происходит упаковка этого значения в тип `object`. Чтобы обратно получить данные в переменную типов `int`, необходимо выполнить распаковку.

Упаковка предполагает преобразование объекта значимого типа (например, типа `int`) к типу `object`. При упаковке значение обертывается в объект типа `System.Object` и сохраняется в управляемой куче. Распаковка, наоборот, предполагает преобразование объекта типа `object` к значимому типу. Упаковка и распаковка ведут к снижению производительности, так как системе надо осуществить необходимые преобразования.

Существует другая проблема - проблема безопасности типов. Так, мы получим ошибку, если неправильно укажем тип в операции преобразования типа, например, в нашем случае это может выглядеть так:

```
pr2.Info = (int)pr2.Info + 1;
```

Мы можем не знать, какой именно объект представляет `Info`, и при попытке получить число в данном случае мы столкнемся с исключением `InvalidCastException`. Причем с исключением мы столкнемся на этапе выполнения программы.

Кроме того, как мы видели с экземпляром `pr3`, можно получить результат другого типа.

§ 3.2. Обобщенные классы

Для решения выше указанных проблем в язык C# была добавлена поддержка обобщенных типов. Обобщенные типы могут использоваться при описании класса. Такие классы сами называются обобщенными. Для обозначения обобщенного типа обычно используется прописная буква. Она в угловых скобках записывается в заголовке класса после имени и затем используется при описании полей и свойств, например,

```
class Person2<T>
{
    private string name;
    private T info;
    public string Name
    {
        set
        {
            name = value;
        }
        get
        {
            return name;
        }
    }
    public T Info
    {
        set
        {
            info = value;
        }
        get
        {
            return info;
        }
```

```
    }
    public Person2(string name, T info)
    {
        this.name = name;
        this.info = info;
    }
}
```

Таким образом, угловые скобки в описании указывают, что класс является обобщенным, а тип T, заключенный в угловые скобки, будет использоваться этим классом. Причем сейчас на этапе описания класса нам неизвестно, что это будет за тип, это может быть любой тип. Поэтому параметр T в угловых скобках еще называется универсальным параметром.

При использовании класса мы должны произвести типизацию, т.е. после названия класса в угловых скобках указывать конкретный тип, например, int или string, причем для разных экземпляров типы могут различаться. Код в процедуре Main() может иметь вид

```
Console.WriteLine("Использование обобщенного класса");
Person2<int> pr6 = new Person2<int>("Иванов", 123);
Person2<string> pr7 =
      new Person2<string>("Петров", "456");
Person2<char> pr8 = new Person2<char>("Сидоров", 'f');
Person2<bool> pr9 = new Person2<bool>("Титов", false);
Person2<object> pr10 =
      new Person2<object>("Алексеев", pr1);
Console.WriteLine("Фамилия {0}, информация {1}", pr6.Name,
      pr6.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr7.Name,
      pr7.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr8.Name,
      pr8.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr9.Name,
      pr9.Info);
Console.WriteLine("Фамилия {0}, информация {1}",
     pr10.Name, pr10.Info);
```

Здесь каждый экземпляр типизирован своим типом.

После выполнения кода получим

```
Использование обобщенного класса
```

```
Фамилия Иванов, информация 123
Фамилия Петров, информация 456
Фамилия Сидоров, информация f
Фамилия Титов, информация False
Фамилия Алексеев, информация Project1.Person1
```

Нетрудно видеть, что результат полностью совпадает с результатом, полученным с использованием поля типа `object`. Отличие состоит в том, что здесь не происходит упаковки вводимых данных для поля `info` в объекты типа `object`. Поэтому требуется меньше ресурсов на выполнение кода.

Фрагмент кода для обработки и вывода ее результата можно записать так:

```
pr6.Info = pr6.Info + 1;
pr7.Info = pr7.Info + 1;
pr8.Info = (char)(pr8.Info + 1);
pr9.Info = !pr9.Info;
pr10.Info = pr10.Info.ToString();
Console.WriteLine("Фамилия {0}, информация {1}", pr6.Name,
     pr6.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr7.Name,
     pr7.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr8.Name,
     pr8.Info);
Console.WriteLine("Фамилия {0}, информация {1}", pr9.Name,
     pr9.Info);
Console.WriteLine("Фамилия {0}, информация {1}",
     pr10.Name, pr10.Info);
```

Здесь мы опустили операции приведения к типу для распаковки. Ее не требуется. Нам пришлось указать явное преобразование к типу `(char)`. В противном случае мы получали ошибку компиляции.

При его выполнении получим

```
Фамилия Иванов, информация 124
Фамилия Петров, информация 4561
Фамилия Сидоров, информация g
Фамилия Титов, информация True
Фамилия Алексеев, информация Project1.Person1
```

Сравнивая с результатом, полученным для поля типа `object`, видим, что отличие имеется для символьного типа. Здесь мы получили символ, что и является корректным.

Если мы попытаемся в конструктор ввести ошибочные данные другого типа, то опять получим ошибку компиляции.

Таким образом, используя обобщенный вариант класса, мы снижаем время на выполнение и количество потенциальных ошибок.

§ 3.3. Использование нескольких универсальных параметров

При описании класса могут использоваться несколько обобщенных типов. Соответствующие универсальные параметры указываются в заголовке класса через запятую. При типизации они могут принимать одинаковые или разные значения.

В предыдущий класс добавим еще один обобщенный тип, который будем использовать при описании поля для хранения идентификатора сотрудника. Получим

```
class Person3<T,P>
{
    private string name;
    private T info;
    private P id;
    public string Name
    {
        set
        {
            name = value;
        }
        get
        {
            return name;
        }
    }
    public T Info
    {
        set
        {
            info = value;
        }
```

```
        get
        {
            return info;
        }
    }
    public P Id
    {
        set
        {
            id = value;
        }
        get
        {
            return id;
        }
    }
    public Person3(string name, T info, P id)
    {
        this.name = name;
        this.info = info;
        this.id = id;
    }
}
```

Применим данный класс. Для этого в метод `Main()` пропишем код

```
Console.WriteLine("\nИспользование нескольких
        параметров");
Person3<int,int> pr11 =
        new Person3<int, int>("Иванов", 123, 101);
Person3<string,int> pr12 =
        new Person3<string, int>("Петров", "456", 202);
Person3<char,string> pr13 =
        new Person3<char, string>("Сидоров", 'f', "a12");

Console.WriteLine("Фамилия {0}, инфо {1}, id {2}",
        pr11.Name, pr11.Info, pr11.Id);
Console.WriteLine("Фамилия {0}, инфо {1}, id {2}",
        pr12.Name, pr12.Info, pr12.Id);
Console.WriteLine("Фамилия {0}, инфо {1}, id {2}",
        pr13.Name, pr13.Info, pr13.Id);
```

Здесь мы для параметра T использовали типы int, string и char, а для параметра P - int и string. Причем для экземпляра pr11 оба параметра принимают одинаковые значения.

При выполнении кода имеем

```
Использование нескольких параметров
Фамилия Иванов, инфо 123, id 101
Фамилия Петров, инфо 456, id 202
Фамилия Сидоров, инфо f, id a12
```

Обработка полей осуществляется по вышерассмотренному принципу.

§ 3.4. Использование обобщенного класса при типизации

При создании экземпляра обобщенного класса нам приходится фиксировать значение универсального параметра, указывая некоторый тип. В качестве выбранного типа может выступать другой обобщенный класс. Причем для этого класса значение его параметра должно быть выбрано.

Рассмотрим пример. Предположим, что проводятся некоторые соревнования между организациями. Опишем класс, включающий в себя два поля. Первое поле для названия организации, а второе содержит некоторую информацию об участнике. В зависимости от типа организации информация может отличаться для разных участников. Для первого поля выберем тип string, а второго используем обобщенный тип. Описание класса будет иметь вид

```
class Game<P>
{
    private string company;
    private P member;
    public string Company
    {
        set
        {
            company = value;
        }
        get
        {
            return company;
```

```
        }
    }
    public P Member
    {
        set
        {
            member = value;
        }
        get
        {
            return member;
        }
    }
    public Game(string company, P member)
    {
        this.company = company;
        this.member = member;
    }
}
```

При использовании этого класса для типизации универсального параметра укажем тип Person2, который сам содержит универсальный параметров. Его в свою очередь типизируем типом int. Запишем код

```
Console.WriteLine("Обобщенный класс как обобщенный тип");
Game<Person2<int>> gm1 = new Game<Person2<int>>("КГАСУ",
       new Person2<int>("Иванов", 123));
Console.WriteLine("Организация участника {0}, фамилия {1},
       информация {2}", gm1.Company, gm1.Member.Name,
       gm1.Member.Info);
```

Здесь мы конструктору класса Game в качестве второго параметра передаем новый создаваемый экземпляр класса Person2.

Выполнение кода даст следующий результат

```
Обобщенный класс как обобщенный тип
Организация участника КГАСУ, фамилия Иванов,
информация 123
```

§ 3.5. Использование обобщенных типов для статических полей

Рассмотрим применение обобщенных типов при работе со статическими полями. Описание статических полей и свойств с универсальными параметрами осуществляется аналогично выше рассмотренным случаям.

Добавим в класс `Person2` статические члены в виде поля для хранения некоторого кода и соответствующего свойства:

```
static private T code;
static public T Code
{
    set
    {
        code = value;
    }
    get
    {
        return code;
    }
}
```

Статические поля не связаны с конкретными экземплярами. Типизация универсального параметра выполняется при инициализации поля, например, при помощи свойства.

Отметим, что указывая разные типы, мы можем одновременно иметь одноименные статические поля с разными значениями, т.е. мы можем получить целый набор значений статического поля разных типов. Приведем пример кода

```
Console.WriteLine("Статические члены");
Person2<int>.Code = 765;
Person2<string>.Code = "Hello!";
Person2<char>.Code = 'w';
Person2<bool>.Code = true;
Console.WriteLine("Статическое поле имеет значение:\n для
int {0} \n для string {1} \n для char {2}
   \n для bool {3}", Person2<int>.Code,
   Person2<string>.Code, Person2<char>.Code,
   Person2<bool>.Code);
```

При выполнении кода получим

```
Статические члены
Статическое поле имеет значение:
 для int 765
 для string Hello!
 для char w
 для bool True
```

Как видим, в данном случае статическое поле принимает набор из четырех значений.

§ 3.6. Обобщенные методы

Кроме обобщенных классов можно также создавать обобщенные методы, которые будут использовать универсальные параметры для обозначения типов параметров. Рассмотрим метод, который меняет между собой значения двух переменных. Его можно записать так:

```
static void f<T>(ref T x, ref T y)
{
    T z = x;
    x = y;
    y = z;
}
```

Здесь параметры могут иметь произвольный тип. При выполнении кода

```
Console.WriteLine("Обобщенный метод");
int a = 5;
int b = 7;
F<int>(ref a, ref b);
Console.WriteLine("Для чисел  a = {0}, b = {1}", a, b);
string u = "Tom";
string v = "Bob";
F<string>(ref u, ref v);
Console.WriteLine("Для строк  u = {0}, v = {1}", u, v);
char s = 'x';
char t = 'y';
```

```
F<char>(ref s, ref t);
Console.WriteLine("Для символов  s = {0}, t = {1}", s, t);
bool p = true;
bool q = false;
F<bool>(ref p, ref q);
Console.WriteLine("Для логического типа p = {0}, q = {1}",
       p, q);
```

получим

```
Обобщенный метод
Для чисел  a = 7, b = 5
Для строк  u = Bob, v = Tom
Для символов  s = y, t = x
Для логического типа p = False, q = True
```

При вызове метода типизируем его определенным типом и передаем ему соответствующие этому типу значения.

§ 3.7. Наследование обобщенных классов

Для обобщенных классов может быть созданы дочерние классы. При этом можно использовать различные варианты наследования.

Предположим, что мы создаем дочерние классы для класса `Person2`. Также предположим, что создаваемые дочерние классы должны иметь дополнительное поле `id` для хранения идентификатора. Таких классов будет несколько. Их будем называть именами `PersonId` с порядковым номером.

Первый вариант заключается в том, что дочерний класс использует тот же самый универсальный параметр, что и родительский класс. Причем этот параметр используется только для наследуемого поля. Код имеет вид

```
class PersonId1<T> : Person2<T>
{
    private int id;
    public int Id
    {
        set
        {
            id = value;
        }
        get
```

```
        {
            return id;
        }
    }
    public PersonId1(string name, T info, int id) :
          base(name, info)
    {
        this.id = id;
    }
}
```

При выполнении операторов

```
Console.WriteLine("\n Использование первого класса");
PersonId1<string> prid1 =
         new PersonId1<string>("Иванов", "123", 321);
Console.WriteLine("До преобразования info = {0}",
         prid1.Info);
prid1.Info = prid1.Info + 1;
Console.WriteLine("После преобразования info = {0}",
         prid1.Info);
```

получим

```
Использование первого класса
До преобразования info = 123
После преобразования info = 1231
```

По результату видим, что поле `info` имеет тип `string`.

В предыдущем варианте новое поле дочернего класса имеет тип `int` и никак не связан с универсальным параметром родительского класса. Можно этот параметр дополнительно использовать и для поля `id`, например, так:

```
class PersonId2<T> : Person2<T>
{
    private T id;
    public T Id
    {
        set
        {
```

```
            id = value;
        }
        get
        {
            return id;
        }
    }
    public PersonId2(string name, T info, T id) :
         base(name, info)
    {
        this.id = id;
    }
}
```

Теперь один и тот же универсальный параметр использован при описании двух свойств. Запишем код в `Main()`

```
Console.WriteLine("\n Использование второго класса");
PersonId2<string> prid2 =
       new PersonId2<string>("Иванов",  "123", "321");
Console.WriteLine("До преобразования info = {0}, id =
{1}",
       prid2.Info, prid2.Id);
prid2.Info = prid2.Info + 1;
prid2.Id = prid2.Id + 1;
Console.WriteLine("После преобразования info = {0},
       id = {1}", prid2.Info, prid2.Id);
```

выполнение которого даст результат

```
Использование второго класса
До преобразования info = 123, id = 321
После преобразования info = 1231, id = 3211
```

Видим, что оба поля имеют тип `string`.

Можно определить используемый тип непосредственно в заголовке дочернего класса следующим образом

```
class PersonId3 : Person2<char>
{
    private string id;
    public string Id
```

```
    {
        set
        {
            id = value;
        }
        get
        {
            return id;
        }
    }
    public PersonId3(string name, char info, string id) :
          base(name, info)
    {
        this.id = id;
    }
}
```

В этом случае дочерний класс является обычным классом и его использование не требует типизации, так как значение универсального параметра уже определено. При выполнении кода

```
Console.WriteLine("\n Использование третьего класса");
PersonId3 prid3 = new PersonId3("Иванов", 'g', "321");
Console.WriteLine("До преобразования info = {0}, id =
{1}",
       prid3.Info, prid3.Id);
prid3.Info = (char)(prid3.Info + 1);
prid3.Id = prid3.Id + 1;
Console.WriteLine("После преобразования info = {0},
       id = {1}", prid3.Info, prid3.Id);
```

ожидаемо получим результат

```
Использование третьего класса
До преобразования info = g, id = 321
После преобразования info = h, id = 3211
```

Фиксируя значение универсального параметра в заголовке дочернего класса, мы можем в дочернем классе ввести новый универсальный параметр, отличный от зафиксированного, который можно использовать при описании нового поля, например, так:

```
class PersonId4<P> : Person2<char>
{
    private P id;
    public P Id
    {
        set
        {
            id = value;
        }
        get
        {
            return id;
        }
    }
    public PersonId4(string name, char info, P id) :
           base(name, info)
    {
        this.id = id;
    }
}
```

Теперь у нас тип поля `info` зафиксирован и является `string`, а для поля `id` использован обобщенный тип. Его мы должны указать при создании экземпляра:

```
Console.WriteLine("\n Использование четвертого класса");
PersonId4<int> prid4 =
        new PersonId4<int>("Иванов", 'g', 321);
Console.WriteLine("До преобразования info = {0}, id =
{1}",
        prid4.Info, prid4.Id);
prid4.Info = (char)(prid4.Info + 1);
prid4.Id = prid4.Id + 1;
Console.WriteLine("После преобразования info = {0},
        id = {1}", prid4.Info, prid4.Id);
```

В этом случае имеем

```
Использование четвертого класса
До преобразования info = g, id = 321
После преобразования info = h, id = 322
```

И наконец-то, к имеющемуся универсальному параметру родительского класса мы можем добавить новый параметр в дочернем классе:

```
class PersonId5<T,P> : Person2<T>
{
    private P id;
    public P Id
    {
        set
        {
            id = value;
        }
        get
        {
            return id;
        }
    }
    public PersonId5(string name, T info, P id) :
        base(name, info)
    {
        this.id = id;
    }
}
```

Таким образом, в дочернем классе используются два параметра. Их надо указать при создании экземпляра:

```
Console.WriteLine("\n Использование пятого класса");
PersonId5<int,string> prid5 =
       new PersonId5<int, string>("Иванов", 123, "321");
Console.WriteLine("До преобразования info = {0},
       id = {1}", prid5.Info, prid5.Id);
prid5.Info = prid5.Info + 1;
prid5.Id = prid5.Id + 1;
Console.WriteLine("После преобразования info = {0},
       id = {1}", prid5.Info, prid5.Id);
```

При выполнении кода получим

```
Использование пятого класса
```

```
До преобразования info = 123, id = 321
После преобразования info = 124, id = 3211
```

Отсюда следует, что поле `info` имеет тип `int`, а поле `id - string`.

§ 3.8. Ограничения обобщений

С помощью универсальных параметров мы можем типизировать обобщенные классы любым типом. Однако иногда возникает необходимость наложения некоторых ограничений на используемый тип. Это может быть связано с требованием наличия некоторых конкретных свойств, методов или допустимых операций. Предположим, что мы хотим в класс `Game`, рассмотренный в § 3.4, добавить метод, выводящий на консоль фамилию участника. Его код может быть таким

```
public void PrName()
{
    Console.WriteLine("Фамилия участника {0}",
         member.Name);
}
```

Однако у нас появится сообщение об ошибке, что свойство `Name` отсутствует. Для поля `member` использован обобщенный тип, и при типизации мы указывали класс `Person2`, который обладает этим свойством. Ошибка же возникает непосредственно при описании самого класса `Game`. Очевидно, для решения проблемы нужно указать в описании класса `Game`, что для типизации необходимо использовать классы, обладающие данным свойством. Это указывается в заголовке класса в виде

```
class имя<параметр> where параметр: ограничение
```

Здесь в качестве ограничения можно указывать имя самого класса.

Опишем класс

```
class Game1<P> where P: Person1
{
    private string company;
    private P member;
    public string Company
    {
        set
```

```
        {
            company = value;
        }
        get
        {
            return company;
        }
    }
    public P Member
    {
        set
        {
            member = value;
        }
        get
        {
            return member;
        }
    }
    public Game1(string company, P member)
    {
        this.company = company;
        this.member = member;
    }
    public void PrName()
    {
        Console.WriteLine("Фамилия участника {0}",
               member.Name);
    }
}
```

Здесь мы для простоты в качестве ограничения указали класс Person1, который является обычным.

При выполнении кода

```
Console.WriteLine("\n Использование ограничений на
классы");
Game1<Person1> gm2 = new Game1<Person1>("КГАСУ",
      new Person1("Иванов", 123));
gm2.PrName();
```

получим

```
Использование ограничений на классы
Фамилия участника Иванов
```

Может возникнуть вопрос: зачем мы используем обобщенный тип, если сразу на него накладываем ограничение? Дело в том, что мы при типизации можем использовать не только сам указанный класс, но и его производные классы. Все они в силу наследственности будут обладать нужными членами и операциями.

Для примера рассмотрим дочерний класс

```
class PersonId : Person1
{
    private int id;
    public int Id
    {
        set
        {
            id = value;
        }
        get
        {
            return id;
        }
    }
    public PersonId(string name, object info, int id) :
           base(name, info)
    {
        this.id = id;
    }
}
```

При выполнении кода

```
Game1<PersonId> gm3 = new Game1<PersonId>("КГАСУ",
      new PersonId("Петров", "345", 567));
gm3.PrName();
```

получим

```
Фамилия участника Петров
```

Ограничения на обобщения могут устанавливаться и для методов. Эти ограничения указываются по аналогии с ограничениями в классах в заголовке метода.

В классе Program опишем метод

```
static void G<T>(T x) where T: Person1
{
    Console.WriteLine("Фамилия {0}, информация {1}",
         x.Name, x.Info);
}
```

Затем в процедуру Main() добавим фрагмент кода

```
Console.WriteLine("\n Использование ограничений на
методы");
Person1 pr14 = new Person1("Николаев", 423);
PersonId pr15 = new PersonId("Михайлов", "543", 678);
G(pr14);
G(pr15);
```

после выполнения которого получим

```
Использование ограничений на методы
Фамилия Николаев, информация 423
Фамилия Михайлов, информация 543
```

Как видим, метод G() поддерживает класс Person1 и его дочерние классы. Попытка использования других значений универсального параметра приводит к ошибке, например, при записи операторов

```
Person2<int> pr16 = new Person2<int>("Николаев", 423);
G(pr16);
```

мы получим ошибку во второй строке ввиду несоответствия типа.

Рассмотрим, какие типы ограничений могут использоваться. Выше мы в ограничении указывали имя некоторого класса и видели, что для типизации мы можем использовать этот класс или его дочерние классы.

Сразу отметим, что нельзя указывать имя структуры. Дело в том, что для структуры нельзя создать дочернюю структуру. Поэтому получим, что в этом случае можно будет использовать только саму эту структуру, т.е. никакого обобщения нет. При описании вместо универсального параметра

можно сразу записать имя этой структуры, и мы получим обычный класс или метод.

С другой стороны, в ограничении можно указывать имя некоторого интерфейса. В этом случае для типизации мы сможем использовать любой класс или структуру, созданные на основе этого интерфейса. Все они будут иметь методы, приведенные в интерфейсе. Это может быть использовано в теле обобщенного класса или метода.

Кроме конкретных классов и интерфейсов могут использоваться стандартные ограничения. Перейдем к ним. Можно в качестве ограничения указать ключевое слово `class`. Это будем означать, что при типизации можно использовать любой класс. Как мы знаем, экземпляры любых классов относятся к ссылочным типам.

Следующий вариант – это указать слово `struct`. В этом случае универсальный параметр в качестве значения может принимать любую структуру, которые определяют значимые типы.

В качестве ограничения можно указать `new()`. В этом случае можно использовать любую структуру и любой класс, который имеет общедоступный (`public`) конструктор без параметра.

Если для универсального параметра задано несколько ограничений, то они должны идти в определенном порядке:

1. Название класса, `class, struct`. Причем мы можем одновременно определить только одно из этих ограничений
2. Название интерфейса
3. `new()`

Если класс использует несколько универсальных параметров, то последовательно можно задать ограничения к каждому из них, например, заголовок класса может иметь вид

```
class Person3<T,P>
   where T: class
   where P: struct
```

Если для обобщенного класса с ограничением создается обобщенный дочерний класс с тем же самым универсальным параметром, то ограничения должны присутствовать и в заголовке дочернего класса. Ограничения записываются после имени родительского класса, и эти ограничения должны быть теми же или более сильными.

Предположим, что в родительском классе указано ограничение `class`, тогда в дочернем классе мы должны использовать в качестве ограничения либо `class`, либо имя некоторого конкретного класса. Здесь же, при необходимости, можно устанавливать ограничение на новый

универсальный параметр дочернего класса. Заголовки классов могут быть записаны так:

```
class Person2<T> where T: class

class PersonId5<T,P> : Person2<T>
   where T: Persion
   where P: struct
```

Ограничения в методах записываются по тем же самым принципам.

Глава 4. Индексаторы

§ 4.1. Описание индексатора

Часто при работе с некоторой совокупностью однотипных элементов бывает удобно обращать к ним не по имени, а по порядковому номеру. Это позволяет упростить процесс их обработки. Индексаторы как раз и позволяют индексировать объекты и обращаться к данным по индексу. Фактически с помощью индексаторов мы можем работать с объектами как с массивами. По форме они напоминают свойства со стандартными блоками `get` и `set`, которые возвращают и присваивают значение. Описание индексатора имеет вид

```
возвращаемый_тип this [параметры]
{
    set { ... }
    get { ... }

}
```

`Возвращаемый_тип` – это тип объектов, которые индексируются, `параметры` определяют количество индексов и их типы. Каждому индексу соответствует один параметр с указанием его типа. В блоке `set` определяется, при указании какого индекса значение какого объекта изменяется. В блоке `get` определяется, при указании какого индекса какой объект возвращается.

Рассмотрим пример. Опишем класс `Student` с полями для хранения фамилии, группы и оценок по предметам математика, физика, химия. Для краткости записи будем использовать автоматические свойства. Также опишем конструктор с параметрами для инициализации полей и метод для вычисления среднего балла студента. Кроме указанных членов, добавим индексатор для полей. Получим следующий код

```
class Student
{
    public string Name { get; set; }
    public int Grup { set; get; }
    public int Mat { set; get; }
    public int Fiz { set; get; }
    public int Him { set; get; }
```

```
public Student(string name, int grup, int mat,
       int fiz, int him)
{
    Name = name;
    Grup = grup;
    Mat = mat;
    Fiz = fiz;
    Him = him;
}
public double Sr_ball() =>
           Math.Round((Mat + Fiz + Him) / 3.0, 2);
public int this [int k]
{
    set
    {
        switch(k)
        {
            case 0:
                Mat = value;
                break;
            case 1:
                Fiz = value;
                break;
            case 2:
                Him = value;
                break;
            default: throw
                 new Exception("Некорректный номер!");
        }
    }
    get
    {
        switch (k)
        {
            case 0:
                return Mat;
            case 1:
                return Fiz;
            case 2:
                return Him;
            default: throw
```

```
                    new Exception("Некорректный номер!");
            }
        }
    }
}
```

Здесь использованы индексы {0,1,2}.

В процедуру Main() запишем операторы

```
Student st1 = new Student("Иванов", 203, 5, 4, 3);
Console.WriteLine(st1[1]);
st1[1] = 5;
Console.WriteLine(st1.Fiz);
```

При их выполнении получим

```
4
5
```

Как видим, используя индексы, можно прочитать и изменить значение соответствующего поля.

Если попытаться использовать другие значения для индекса, то мы получим исключение с сообщением "Некорректный номер!".

Мы уже говорили, что при индексировании с объектами можно работать как с массивами, но надо иметь в виду, что они не представляют собой массивы, и, например, нельзя для них использовать свойства и методы класса System.Array.

Индексатор получает набор индексов в виде параметров. Однако индексы необязательно должны представлять тип int, могут использоваться и другие типы.

§ 4.2. Перегрузка индексатора

Как и для методов, для индексаторов можно определять перегрузки. Они должны отличаться количеством, типом или последовательностью параметров. Добавим в класс Student следующий фрагмент

```
public int this[string s]
{
    set
    {
        switch (s)
```

```
        {
            case "Математика":
                Mat = value;
                break;
            case "Физика":
                Fiz = value;
                break;
            case "Химия":
                Him = value;
                break;
            default:
                throw new Exception("Некорректный
                            индекс!");
        }
    }
    get
    {
        switch (s)
        {
            case "Математика":
                return Mat;
            case "Физика":
                return Fiz;
            case "Химия":
                return Him;
            default:
                throw new Exception("Некорректный
                      индекс!");
        }
    }
}
```

Мы получили две перегрузки индексатора полей. Первая перегрузка использует индексы типа `int`, вторая – типа `string`. Для иллюстрации применения индексатора добавим в метод `Main()` фрагмент

```
Console.WriteLine("Оценка по математике {0}",
                st1["Математика"]);
st1["Математика"] = 4;
Console.WriteLine("Оценка по математике {0}", st1[0]);
```

при выполнении которого получим

```
Оценка по математике 5
Оценка по математике 4
```

Как видим, мы можем обращать к полю, используя индексы двух типов. Естественно, сохраняется возможность обращения к полю и по имени.

Рассмотрим сочетание массива с индексатором. Выполняя код

```
Student st1 = new Student("Иванов", 203,5,4,3);
Student st2 = new Student("Петров", 204,3,4,4);
Student st3 = new Student("Сидоров", 201, 4, 5, 4);
Student st4 = new Student("Алексеев", 202, 5, 5, 4);
Student st5 = new Student("Титов", 201, 4, 5, 5);
Student[] stud1 = new Student[] {st1,st2,st3,st4,st5};
double s_mat = 0;
for (int k = 0; k < stud1.Length; k++)
    s_mat += stud1[k]["Математика"];
s_mat /= stud1.Length;
Console.WriteLine("Средний балл по математике равен {0}",
         Math.Round(s_mat,2));
```

получим

```
Средний балл по математике равен 4,2
```

Здесь мы создали пять экземпляров класса `Student`, из них составили массив и вычислили средний балл по математике. По виду оператора, стоящего в теле цикла, кажется, что мы используем массив массивов, но это не так. Первый индекс действительно связан с массивом, а второй – с индексатором.

§ 4.3. Индексирование компонентов поля

В рассмотренном примере мы индексы использовали для доступа к полям типа `int`. При этом сами значения индексов были никак не связаны со значениями полей. Однако поля могут иметь более сложную структуру, например, быть массивами некоторых объектов, и индексы могут связываться со значениями некоторых полей соответствующих членов массива.

Рассмотрим пример. Опишем класс Students, который имеет поле, представляющее собой массив элементов типа Student. Кроме индексатора типа int, введем индексатор, который позволит получать объект типа Student из этого поля по значению Name, т.е. мы указываем фамилию студента и получаем доступ к его данным. Класс можно описать так:

```
class Students
{
    private Student[] stud;
    public Students(Student[] stud)
    {
        this.stud = stud;
    }
    public Student this[int k]
    {
        set { stud[k] = value; }
        get { return stud[k];  }
    }
    public Student this[string s]
    {
        get
        {
            foreach (Student st in stud)
            {
                if (st.Name == s)
                    return st;
            }
            throw new Exception("Нет такого студента!");

        }
    }
}
```

Отметим, что мы у перегрузки индексатора типа string описали только блок get. Поэтому эта перегрузка только для чтения. Для перегрузки индексатора типа int номер индексатора совпадает с номером элемента в массиве. Также обратим внимание, что в этом классе отсутствуют свойства, и поле имеет модификатор доступа private. Поэтому доступ к данному полю возможно только с использованием индексов.

Добавим в процедуру `Main()` фрагмент

```
Students studs = new Students(stud1);
for (int k = 0; k < stud1.Length; k++)
    Console.WriteLine("Студент {0}, группа {1}",
        studs[k].Name, studs[k].Grup);
Console.WriteLine("Студент {0}, средний балл {1}",
        studs["Петров"].Name, studs["Петров"].Sr_ball);
```

При выполнении имеем

```
Студент Иванов, группа 203
Студент Петров, группа 204
Студент Сидоров, группа 201
Студент Алексеев, группа 202
Студент Титов, группа 201
Студент Петров, средний балл 3,67
```

Мы были вынуждены в операторе цикла записать `stud1.Length`. Мы не можем использовать `studs.Length`. Так как `studs` не является массивом и не имеет данного свойства в отличие от `stud1`.

Если мы при использовании индекса типа `string` укажем несуществующее значение `Name`, то будет сгенерировано исключение с соответствующим сообщением.

Индексатор может использовать несколько параметров. Допустим, у нас есть класс, в котором хранятся данные в виде матрицы:

```
class Matrix1
{
    private int[,] m = new int[,]{{1,2,3},{4,5,6}};
    public int this[int i, int j]
    {
        set { m[i,j] = value; }
        get { return m[i,j]; }
    }
}
```

Теперь для определения индексатора используются два индекса - `i` и `j`. И в программе мы уже должны обращаться к объекту, используя два индекса. Выполним код

```
Matrix1 mat = new Matrix1();
mat[1, 2] = 10;
Console.WriteLine("Первый = {0}, последний = {1}",
      mat[0, 0], mat[1, 2]);
```

Получим

```
Первый = 1, последний = 10
```

Следует учитывать, что индексатор не может быть статическим и применяется только к экземпляру класса. Однако индексаторы могут быть виртуальными и абстрактными и могут переопределяться в производных классах.

Глава 5. Коллекции

§ 5.1. Список List<T>

В языке C#, как и во многих языках программирования, для работы с некоторой совокупностью данных обычно используют массивы. Однако здесь для этих целей могут использоваться и коллекции. Основным отличием от массива является то, что количество членов, входящих в коллекцию не фиксировано. Мы всегда можем добавлять новые элементы и удалять существующие. Кроме того, коллекции реализуют стандартные структуры данных со своими свойствами и методами, которые могут пригодиться для решения различных специальных задач. Большая часть классов коллекций содержится в пространстве имен `System.Collections.Generic`.

Класс `List<T>` из пространства имен `System.Collections.Generic` представляет простейший список однотипных объектов. Класс `List` типизируется типом, объекты которого будут храниться в списке.

Мы можем создать пустой список:

```
List<string> people = new List<string>();
```

В данном случае объект `List` типизируется типом `string`. Это значит, что хранить в этом списке мы можем только строки.

Заметим, что при создании списка мы можем указать в круглых скобках начальную емкость списка, например, так:

```
List<string> people = new List<string>(16);
```

Указание начальной емкости списка позволяет в будущем увеличить производительность и уменьшить издержки на выделение памяти при добавлении элементов. Так как динамическое добавление в список может приводить к дополнительному выделению памяти, а это снижает производительность. Если же мы знаем, что список не будет превышать некоторый размер, то мы можем передать этот размер в качестве емкости списка и избежать дополнительных выделений памяти.

Начальную емкость также можно установить с помощью свойства `Capacity`, которое имеется у класса `List`.

При создании списка можно сразу инициализировать его начальными значениями. В этом случае элементы списка помещаются после вызова конструктора в фигурных скобках

```
List<string> people =
       new List<string>(){"Иванов", "Петров", "Сидоров"};
```

В данном случае в список помещаются три строки.

Также можно при создании списка инициализировать его элементами из другой коллекции, например, другого списка:

```
var members = new List<string>(people);
```

Можно совместить оба способа:

```
var members = new List<string>(people){"Алексеев"};
```

Здесь в списке `members` четыре элемента `{"Иванов", "Петров", "Сидоров", "Алексеев"},` три из которых добавляются из списка `people` и один элемент задается при инициализации.

Подобным образом можно работать со списками других типов, например, типа `Student`, определенного в предыдущей главе. Код может иметь вид

```
Student st1 = new Student("Иванов", 203, 5, 4, 3);
Student st2 = new Student("Петров", 204, 3, 4, 4);
Student st3 = new Student("Сидоров", 201, 4, 5, 4);
Student st4 = new Student("Алексеев", 202, 5, 5, 4);
Student st5 = new Student("Титов", 201, 4, 5, 5);
List<Student> list1 =
          new List<Student> {st1, st2, st3};
```

Здесь мы создали список из трех элементов типа `Student`.

Как и в массивах, к членам списка можно обратиться при помощи индекса, например, при выполнении оператора

```
Console.WriteLine("Студент {0}, средний балл {1}",
        list1[2].Name, list1[2].Sr_ball);
```

получим

```
Студент Сидоров, средний балл 4,33
```

Обратим внимание, что нумерация начинается с нуля.

Добавить новые элементы в список можно при помощи метода `Add()`. Здесь параметром является добавляемый элемент. Он будет добавлен в конец списка. Если же мы хотим некоторый элемент вставить под определенным номером, то следует воспользоваться методом `Insert()`. У этого метода два параметра. Первый параметр соответствует номеру, а второй – вставляемому элементу.

Добавим в процедуру `Main()` фрагмент

```
list1.Add(st4);
list1.Insert(2, st5);
foreach (var x in list1)
    Console.WriteLine("Студент {0} средний балл {1}",
              x.Name, x.Sr_ball());
```

Здесь мы для перебора элементов списка использовали оператор `foreach`. Выполнение фрагмента даст результат

```
Студент Иванов средний балл 4
Студент Петров средний балл 3,67
Студент Титов средний балл 4,67
Студент Сидоров средний балл 4,33
Студент Алексеев средний балл 4,67
```

Отсюда нетрудно видеть порядок расположения объектов.

При помощи метода `Reverse()` можно изменить порядок элементов на противоположный. Действительно при выполнении операторов

```
list1.Reverse();
foreach (var x in list1)
    Console.WriteLine("Студент {0} средний балл {1}",
              x.Name, x.Sr_ball());
```

получим

```
Студент Алексеев средний балл 4,67
Студент Сидоров средний балл 4,33
Студент Титов средний балл 4,67
Студент Петров средний балл 3,67
Студент Иванов средний балл 4
```

Выше мы для перебора элементов списка использовали оператор `foreach`. Можно с помощью свойства `Count` получить длину списка и

воспользоваться оператором for. Поэтому последний фрагмент кода можно заменить равносильным фрагментом

```
list1.Reverse();
for(int k = 0; k < list1.Count; k++)
    Console.WriteLine("Студент {0} средний балл {1}",
              list1[k].Name, list1[k].Sr_ball());
```

Заметим, что свойством Count обладают все коллекции.

Ранее мы рассмотрели добавление новых элементов в список. Сейчас рассмотрим их удаление. При помощи метода Remove() можно удалить некоторый конкретный объект из списка. В качестве параметра выступает идентификатор объекта. Если несколько одинаковых членов, подлежащих удалению, то удаляется первый из них. Этот метод является функцией типа bool. При успешном удалении возвращается значение true, и значение false в противном случае.

Можно удалить элемент, указывая его номер в списке. Для этого используется метод RemoveAt(). Параметром здесь служит номер удаляемого элемента. Добавим операторы

```
if(list1.Remove(st2))
    Console.WriteLine("Элемент успешно удален");
else
    Console.WriteLine("Элемент не удален");

if(list1.Remove(st2))
    Console.WriteLine("Элемент успешно удален");
else
    Console.WriteLine("Элемент не удален");

list1.RemoveAt(2);
for (int k = 0; k < list1.Count; k++)
    Console.WriteLine("Студент {0} средний балл {1}",
          list1[k].Name, list1[k].Sr_ball());
```

При их выполнении получим

```
Элемент успешно удален
Элемент не удален
Студент Алексеев средний балл 4,67
Студент Сидоров средний балл 4,33
Студент Иванов средний балл 4
```

Здесь мы студента Титова удалили из списка при помощи метода `RemoveAt()`, а студента Петрова – при помощи метода `Remove()`, который находился как логическое выражение непосредственно оператора условия. Этот оператор условия мы написали дважды. Поэтому в первый раз мы получили сообщение об успешном удалении элемента, а во второй раз – о том, что элемент не удален. Так как он уже отсутствовал в списке.

При помощи метода `Contains()` можно определить, входит ли записанный в качестве параметра объект в список или нет. Операторы

```
Console.WriteLine("st1 содержится {0}",
         list1.Contains(st1));
Console.WriteLine("st2 содержится {0}",
         list1.Contains(st2));
```

дадут результат

```
st1 содержится True
st2 содержится False
```

Можно найти индексы элементов по значению. Для этого используются методы `IndexOf()` и `LastIndexOf()`. Значения передаются в качестве параметра. Первый из методов возвращает номер первого вхождения указанного значения в список, а второй – последнего вхождения. Для иллюстрации методов добавим фрагмент

```
list1.Add(st3);
list1.Add(st2);
list1.Add(st3);
list1.Add(st5);
Console.WriteLine("Первое вхождение {0} под номером {1}",
        st3.Name,list1.IndexOf(st3));
Console.WriteLine("Последнее вхождение {0} под номером
{1}",
        st3.Name, list1.LastIndexOf(st3));
for (int k = 0; k < list1.Count; k++)
    Console.WriteLine("Студент {0}", list1[k].Name);
```

При выполнении фрагмента имеем

```
Первое вхождение Сидоров под номером 1
Последнее вхождение Сидоров под номером 5
Студент Алексеев
```

```
Студент Сидоров
Студент Иванов
Студент Сидоров
Студент Петров
Студент Сидоров
Студент Титов
```

Мы специально вывели список из фамилий студентов, чтобы проверить полученный результат.

И наконец-то, для всех коллекций доступен метод `Clear()`, который удаляет все элементы списка, и мы получаем пустой список. При выполнении операторов

```
list1.Clear();
Console.WriteLine("Список содержит {0} элементов",
          list1.Count);
```

получим строку

```
Список содержит 0 элементов
```

§ 5.2. Очередь Queue<T>

Коллекция **Queue<T>** представляет обычную очередь, которая работает по принципу FIFO ("первый вошел - первый вышел"). Очередь представляет собой упорядоченную последовательность, но отсутствует нумерация элементов, т.е. элементы в очереди расположены в некоторой последовательности, но у них нет номеров. Поэтому в отличие от списка мы не можем обращаться к элементам по его номеру, используя индекс, и для перебора очереди можно использовать цикл `foreach`. В качестве иллюстрации этой коллекции можно привести, например, очередь в кассу.

Для создания очереди можно использовать один из трех ее конструкторов. Прежде всего, можно создать пустую очередь:

```
Queue<string> people = new Queue<string>();
```

При создании пустой очереди можно указать емкость очереди:

```
Queue<string> people = new Queue<string>(16);
```

Также можно инициализировать очередь элементами из другой коллекции или массивом:

```
string[] members = {"Иванов", "Петров", "Сидоров"};
Queue<string> people = new Queue<string>(members);
```

Как и любая коллекция, очередь обладает свойством `Count` для получения количества элементов в очереди и методом `Clear()` для очистки очереди. Здесь также доступен метод `Contains()`, позволяющий выяснить, входит ли указанный в качестве параметра элемент в очередь или нет.

Рассмотрим другие методы. Добавить новый элемент можно только в конец очереди. Для этого предназначен метод `Enqueue()`. Здесь добавляемый элемент передается в качестве параметра.

Элементы можно извлекать только из начала очереди. Если применить метод `Dequeue()`, то мы получим первый элемент очереди, причем далее этот элемент уже будет отсутствовать в очереди. Если же применить метод `Peek()`, то мы также получим первый элемент, но он сохранится в очереди.

Рассмотрим пример. Сначала создадим массив объектов типа `Student` и на его основе создадим очередь. Для проверки элементов, входящих в очередь, переберем их. Затем добавим в очередь еще один элемент и опять переберем их. Потом получим первый элемент очереди, не извлекая его оттуда. После этого проверим, что элемент `st1`, который был первым в очереди, остался в очереди. И наконец-то, по одному извлечем все элементы из очереди и вычислим их средние баллы. После каждого действия для проверки будем выводить количество элементов в очереди. Соответствующий код будет иметь вид

```
//Создаем очередь на основе массива
Student[] studs = { st1, st2, st3, st4 };
Queue<Student> queue1 = new Queue<Student>(studs);

//Выводим количество элементов в коллекции
Console.WriteLine("В очереди  {0} элементов",
         queue1.Count);
Console.WriteLine();

//Переберем элементы очереди
foreach (var x in queue1)
    Console.WriteLine("Студент {0} ", x.Name);
Console.WriteLine();

//Добавляем элемент
```

```
queue1.Enqueue(st5);

//Выводим количество элементов в коллекции
Console.WriteLine("В очереди  {0} элементов",
        queue1.Count);
Console.WriteLine();

//Переберем элементы очереди
foreach (var x in queue1)
    Console.WriteLine("Студент {0} ", x.Name);
Console.WriteLine();

//Получим первый элемент, не извлекая из очереди
Student st = queue1.Peek();
Console.WriteLine("Получен студент {0} без удаления
        из очереди", st.Name);
Console.WriteLine();

//Проверяем, что элемент st1 содержится в очереди
Console.WriteLine("st1 содержится {0}",
        queue1.Contains(st1));
Console.WriteLine();

//Выводим количество элементов в коллекции
Console.WriteLine("В очереди  {0} элементов",
        queue1.Count);
Console.WriteLine();

//Извлекаем элементы из очереди
while (queue1.Count>0)
{
    var x = queue1.Dequeue();
    Console.WriteLine("Студент {0} средний балл {1}",
         x.Name, x.Sr_ball());
}
Console.WriteLine();

//Выводим количество элементов в коллекции
Console.WriteLine("В очереди  {0} элементов",
         queue1.Count);
```

Этот фрагмент добавим в процедуру `Main()`, использованную в предыдущем параграфе, и после выполнения получим

```
В коллекции 4 элементов

Студент Иванов
Студент Петров
Студент Сидоров
Студент Алексеев

В коллекции 5 элементов

Студент Иванов
Студент Петров
Студент Сидоров
Студент Алексеев
Студент Титов

Получен студент Иванов без удаления из коллекции

st1 содержится True

В коллекции 5 элементов

Студент Иванов средний балл 4
Студент Петров средний балл 3,67
Студент Сидоров средний балл 4,33
Студент Алексеев средний балл 4,67
Студент Титов средний балл 4,57

В коллекции 0 элементов
```

Как видим, мы все элементы извлекли, и очередь пуста.

§ 5.3. Стек Stack<T>

Класс `Stack<T>` представляет коллекцию, которая использует алгоритм LIFO ("последний вошел - первый вышел"). При такой организации каждый следующий добавленный элемент помещается поверх предыдущего. Извлечение из коллекции происходит в обратном порядке - извлекается тот элемент, который находится выше всех в стеке. Стек -

довольна часто встречаемая структура данных в реальной жизни. Примером может служить стопка некоторых предметов. Если нам нужно разобрать стопку, то мы вначале убираем те предметы, которые были добавлены в стопку последними.

Стеки очень близки к очередям. Их можно рассматривать как разновидность очереди. Поэтому для них похожими являются методы и принципы их использования.

Для стека также можно создать пустую коллекцию, указать начальную емкость или создать стек на основе другой коллекции или массива. Соответствующие операторы могут иметь вид

```
Stack<string> people = new Stack <string>();:

Stack<string> people = new Stack<string>(16);

string[] members = {"Иванов", "Петров", "Сидоров"};
Stack<string> people = new Stack<string>(members);
```

Для этой коллекции тоже доступны свойство `Count` и методы `Clear()` и `Contains()`. Рассмотрим методы для добавления элементов и извлечения их.

Для добавления элементов используется метод `Push()`. При этом элемент будет добавляться в начало коллекции. Метод `Pop()` извлекает и возвращает первый элемент из стека, метод `Peek()` просто возвращает первый элемент из стека без его удаления.

Для иллюстрации работы со стеком выполним действия, аналогичные рассмотренным для очереди. Соответствующий код имеет вид

```
//Создаем стек на основе массива
Stack<Student> stack1 = new Stack<Student>(studs);

//Выводим количество элементов в коллекции
Console.WriteLine("В коллекции  {0} элементов",
       stack1.Count);
Console.WriteLine();

//Переберем элементы коллекции
foreach (var x in stack1)
    Console.WriteLine("Студент {0} ", x.Name);
Console.WriteLine();
```

```
//Добавляем элемент
stack1.Push(st5);

//Выводим количество элементов в коллекции
Console.WriteLine("В коллекции  {0} элементов",
      stack1.Count);
Console.WriteLine();

//Переберем элементы коллекции
foreach (var x in stack1)
    Console.WriteLine("Студент {0} ", x.Name);
Console.WriteLine();

//Получим первый элемент, не извлекая из коллекции
Student st = stack1.Peek();
Console.WriteLine("Получен студент {0} без удаления из
       коллекции", st.Name);
Console.WriteLine();

//Проверяем, что элемент st5 содержится в коллекции
Console.WriteLine("st5 содержится {0}",
      stack1.Contains(st5));
Console.WriteLine();

//Выводим количество элементов в коллекции
Console.WriteLine("В коллекции {0} элементов",
      stack1.Count);
Console.WriteLine();

//Извлекаем элементы из коллекции
while (stack1.Count>0)
{
    var x = stack1.Pop();
    Console.WriteLine("Студент {0} средний балл {1}",
        x.Name, x.Sr_ball());
}

Console.WriteLine();
//Выводим количество элементов в коллекции
Console.WriteLine("В коллекции  {0} элементов",
       stack1.Count);
```

При выполнении фрагмента кода имеем

```
В коллекции 4 элементов

Студент Алексеев
Студент Сидоров
Студент Петров
Студент Иванов

В коллекции 5 элементов

Студент Титов
Студент Алексеев
Студент Сидоров
Студент Петров
Студент Иванов

Получен студент Титов без удаления из коллекции

st5 содержится True

В коллекции 5 элементов

Студент Титов средний балл 4,57
Студент Алексеев средний балл 4,67
Студент Сидоров средний балл 4,33
Студент Петров средний балл 3,67
Студент Иванов средний балл 4

В коллекции 0 элементов
```

Сравнивая эти строки с полученным результатом в предыдущем параграфе, видим, что элементы при выводе расположены в обратной последовательности. Уже при создании стека на основе массива элементы располагаются в обратной последовательности, т.е. последний элемент массива оказался первым элементом стека.

§ 5.4. Словарь Dictionary<K, V>

Еще один распространенный тип коллекции представляют словари. Словарь хранит объекты, которые представляют пару ключ-значение. Эту коллекцию можно рассматривать как некоторый аналог списка. В списке мы могли получить доступ к элементу по его номеру. Здесь же мы получаем доступ к значению по ключу, т.е. номер в списке играет роль ключа в словаре. Однако имеются принципиальные отличая. Номер принимает только целые неотрицательные значения, а ключ может иметь любой тип. Кроме того, номер элемента в списке может измениться при вставке или удалении других элементов, а ключ всегда связан с данным значением. Класс словаря `Dictionary<K, V>` типизируется двумя типами: параметр `K` представляет тип ключей, а параметр `V` предоставляет тип значений.

Рассмотрим создание словаря. Можно создать пустой словарь, например, так:

```
Dictionary<int, string> people =
             new Dictionary<int, string>();
```

Здесь словарь `people` в качестве ключей принимает значения типа `int`, а в качестве значений - `string`.

При создании словаря его сразу же можно инициализировать значениями по аналогии с инициализацией двумерного массива:

```
var people = new Dictionary<int, string>()
{
    { 5, "Иванов"},
    { 3, "Петров"},
    { 11, "Сидоров"}
};
```

Отсюда имеем, что значению `"Иванов"` соответствует ключ `5` и так далее.

Можно использовать другую запись этого оператора:

```
var people = new Dictionary<int, string>()
{
    [5] = "Иванов",
    [3] = "Петров",
    [11] = "Сидоров"
};
```

Получим тот же самый результат.

Для иллюстрации выполнения рассматриваемых методов рассмотрим следующий словарь

```
Dictionary<string, Student> dictionary1 =
         new Dictionary<string, Student>
{
    ["КГАСУ"] = st1,
    ["КФУ"] = st2,
    ["КГТУ"] = st3
 };
```

Здесь мы в качестве ключа используем название учебного заведения, а значения – объекты типа `Student`, созданные ранее.

Элементы словаря обладают свойствами `Key` и `Value`. Первый из них возвращает ключ, а второе – значение элемента. Для перебора словаря можно использовать оператор цикла `foreach`. Также доступными остаются свойство `Count` и метод `Clear()`.

Добавим в процедуру `Main()` фрагмент кода

```
Console.WriteLine();
Console.WriteLine("В коллекции {0} элементов",
        dictionary1.Count);
foreach (var x in dictionary1)
{
    Console.WriteLine("Ключ {0} студент {1}",
         x.Key, x.Value.Name);
}
```

при выполнении которого имеем строки

```
В коллекции 3 элементов
Ключ КГАСУ студент Иванов
Ключ КФУ студент Петров
Ключ КГТУ студент Сидоров
```

Рассмотрим вопрос добавления новых элементов в словарь. Как обычно, можно использовать метод `Add()`, который в этом случае принимает два параметра, соответственно, для ключа и значения. Добавим в коллекцию два элемента при помощи операторов

```
dictionary1.Add("НГПУ", st4);
```

```
dictionary1.Add("КЮИ", st5);
```

После этих операторов опять проверим количество элементов в коллекции и выведем фамилии студентов. Получим

```
В коллекции 5 элементов
Ключ КГАСУ студент Иванов
Ключ КФУ студент Петров
Ключ КГТУ студент Сидоров
Ключ НГПУ студент Алексеев
Ключ КЮИ студент Титов
```

Заметим, если мы попытаемся добавить элемент с уже существующим ключом, то возникнет исключение. Все ключи должны различаться.

Ранее мы получили значение при помощи свойства `Value` соответствующего элемента. Можно получить доступ к значению и при помощи ключа, например, так:

```
Console.WriteLine("Студент {0} средний балл {1}",
   dictionary1["КФУ"].Name, dictionary1["КФУ"].Sr_ball());
```

Выполнение оператора приведет к результату

```
Студент Петров средний балл 3,67
```

Ключ можно использовать и для изменения словаря. Добавим оператор

```
dictionary1["КФУ"].Name = "Николаев";
```

и теперь при выполнении предыдущего оператора получим

```
Студент Николаев средний балл 3,67
```

Рассмотрим удаление из словаря. Для этого можно использовать метод `Remove()`. В качестве параметра ему передается ключ удаляемого элемента. Этот метод является функцией, и она возвращает `true` при удачном удалении и `false` – в противном случае.

Добавим операторы

```
if (dictionary1.Remove("КФУ"))
```

```
    Console.WriteLine("Элемент удален!");
else
    Console.WriteLine("Элемент  не удален!");
Console.WriteLine("В коллекции  {0} элементов",
          dictionary1.Count);
```

при выполнении которых получим

```
Элемент удален!
В коллекции 4 элементов
```

Можно определить, входит ли некоторый элемент в словарь. Для этого предназначены методы `ContainsKey()` и `ContainsValue()`. Первый из них в качестве параметра получает ключ проверяемого элемента, а второй – его значение. Если элемент принадлежит словарю, то возвращается `true`, и `false` в противном случае. В качестве примера приведем операторы

```
Console.WriteLine("Элемент содержится в коллекции {0}",
       dictionary1.ContainsKey("МГУ"));
Console.WriteLine("Элемент содержится в коллекции {0}",
       dictionary1.ContainsValue(st5));
```

Результат применения будет иметь вид

```
Элемент содержится в коллекции False
Элемент содержится в коллекции True
```

Как видим, элемент со значением `st5` содержится в коллекции, а элемента с ключом `"МГУ"` нет.

Глава 6. Делегаты и события

§ 6.1. Понятие делегата

Делегаты можно рассматривать как функциональный тип данных, т.е. в них могут храниться ссылки на методы. Делегаты могут использоваться, например, когда требуется выбрать конкретный метод из некоторой коллекции уже на этапе выполнения программы, или при создании событий.

Рассмотрим принципы использования делегатов. Вначале надо описать делегат. Для этого используется ключевое слово `delegate`, указываются тип возвращаемого значения, имя делегата и в скобках типы каждого из параметров с указанием их статусов. Именно на такие методы может указывать данный делегат. Описание делегата может находиться на уровне пространства имен или внутри некоторого класса. От этого зависит область видимости делегата для его использования.

Затем описываются конкретные методы, на которые может указывать делегат. Далее вводится переменная типа объявленного делегата и создается соответствующий экземпляр. Можно просто переменной присвоить имя метода или создать экземпляр при помощи конструктора, указав имя метода в качестве параметра. При этом переменная станет ссылкой на тело данного метода. Теперь с переменной можно работать как с его именем.

Заметим, что созданный экземпляр делегата обычно тоже называют делегатом. Поэтому делегат можно понимать в двух смыслах, а именно, как само описание, так и соответствующий экземпляр.

Рассмотрим пример. На уровне пространства имен опишем делегат `Del` типа `void` без параметров и класс `Test` с двумя методами соответствующего типа, один из которых статический. В классе `Program` также опишем аналогичный статический метод. В процедуре `Main()` введем переменную типа описанного делегата и будем последовательно присваивать имена методов и вызывать их. При этом для первого метода используем конструктор, хотя это не обязательно. Имеем код

```
delegate void Del();

class Program
{

    static void Main(string[] args)
    {
```

```
        Del del1 = new Del(f1);
        del1();
        del1 = Test.f2;
        del1();
        del1 = new Test().f3;
        del1();
    }
    static void f1() => Console.WriteLine("Привет!");
}
class Test
{
    public static void f2() =>
          Console.WriteLine("Как дела?");
    public void f3() => Console.WriteLine("До свидания!");
}
```

При его выполнении получим результат

```
Привет!
Как дела?
До свидания!
```

Обратим внимание на операторы присваивания в процедуре `Main()`. Для первого метода мы использовали конструктор, хотя это было не обязательно. Могли просто записать

```
Del del1 = f1;
```

При присваивании `f2`, как и при использовании любого статического члена другого класса, мы должны указать имя этого класса. Метод `f3` является динамическим. Поэтому для его использования нужно было создать экземпляр класса `Test` при помощи его конструктора.

Выше мы использовали делегат типа `void` без параметров. Совершенно аналогично используются делегаты других типов и с параметрами. Добавим описание другого делегата

```
delegate int Fun(int x, int y);
```

В класс `Program` добавим методы

```
static int g1(int a, int b) => a + b;
```

```
static int g2(int a, int b) => a * b;
```

и в процедуру `Main()` запишем код

```
Fun fun1 = g1;
Console.WriteLine(fun1(3, 4));
fun1 = g2;
Console.WriteLine(fun1(3, 4));
```

При его выполнении получим результат

```
7
12
```

Как видим, при первом вызове делегата была найдена сумма, а при втором – произведение использованных параметров.

При описании делегатов для обозначения типов можно использовать универсальные параметры по аналогии с обобщенными методами. В этом случае получим обобщенные делегаты. Для иллюстрации приведем делегат

```
delegate T Pr<T, P, K>(P x, K y);
```

Используя операторы

```
Pr<int, int, int> pr1 = g1;
Console.WriteLine(pr1(3, 4));
```

убедимся, что опять будет найдена сумма использованных чисел, т.е. выполняется метод `g1()`.

§ 6.2. Добавление методов в делегат и их удаление из делегата

В примерах, рассмотренных выше, переменная указывала на один метод. Имеется возможность, чтобы переменная одновременно содержала ссылки на некоторую совокупность методов. Для добавления метода используется операция `+=`. При этом метод добавляется в список вызова. При вызове делегата все методы из списка последовательно вызываются.

Методы можно и удалять из списка вызова. Для этого используется операция `-=`. Если удаляемый метод уже отсутствует в списке, то операция будет проигнорирована.

Выполним код

```
Del del1 = f1;

Console.WriteLine("Добавление методов");
del1 += Test.f2;
Test st = new Test();
del1 += st.f3;
del1 += f1;
del1 += new Test().f3;
del1();
Console.WriteLine();

Console.WriteLine("Удаление методов");
del1 -= st.f3;
del1 -= Test.f2;
del1 -= Test.f2;
del1 -= new Test().f3;
del1();
```

и получим

```
Добавление методов
Привет!
Как дела?
До свидания!
Привет!
До свидания!

Удаление методов
Привет!
Привет!
До свидания!
```

Как видим, у нас четыре операции добавления методов. Поэтому с учетом начального значения делегата получаем в списке пять методов, и все они были выполнены. У нас также четыре операции удаления, но в списке остались три метода. Это связано с тем, что, во-первых, мы повторно пытались удалить метод `Test.f2`, который уже отсутствовал в списке вызова, и, во-вторых, в операторе

```
del1 -= new Test().f3;
```

создается другой экземпляр класса `Test`, отличающийся от экземпляра, созданного в операторе

```
del1 += new Test().f3;
```

Поэтому эти методы считаются разными.

При удалении следует учитывать, что если делегат содержит несколько ссылок на один и тот же метод, то операция `-=` начинает поиск с конца списка вызова делегата и удаляет только первое найденное вхождение, например, код

```
Del del1 = f1;
Console.WriteLine("Добавление методов");
del1 += Test.f2;
del1 += f1;
del1 += new Test().f3;
del1 += f1;
Test st = new Test();
del1 += st.f3;
del1 += f1;
del1();

Console.WriteLine();
Console.WriteLine("Удаление методов");
del1 -= f1;
del1 -= f1;
del1();
```

приведет к результату

```
Добавление методов
Привет!
Как дела?
Привет!
До свидания!
Привет!
До свидания!
Привет!

Удаление методов
Привет!
```

```
Как дела?
Привет!
До свидания!
До свидания!
```

Как видим, отсутствуют два последних применения метода `f1()`.

При удалении может возникнуть ситуация, что мы удалим все методы, и делегат получит значение `null`. При попытке выполнить такой делегат возникнет исключение. Поэтому желательно перед выполнением делегата проверять его на неравенство значению `null`. Приведем фрагмент кода

```
Del del1 = f1;
del1 += f1;
if(del1==null)
    Console.WriteLine("null");
else
{
    del1();
    Console.WriteLine();
}

del1 -= f1;
if (del1 == null)
    Console.WriteLine("null");
else
{
    del1();
    Console.WriteLine();
}

del1 -= f1;
if (del1 == null)
    Console.WriteLine("Значение null");
else
{
    del1();
Console.WriteLine();
}
```

при выполнении которого получим

```
Привет!
Привет!

Привет!

Значение null
```

Здесь мы при каждом выполнении делегата проверяли его неравенство значению null.

Выше наши примеры были связаны с делегатами типа void. Отдельно нужно рассмотреть добавление функций. Выполним операторы

```
Fun fun1 = g1;
fun1 += g2;
Console.WriteLine(fun1(3, 4));
```

и получим

```
12
```

т.е. ответ соответствует результату выполнения метода g2(), в котором вычисляется произведение параметров. Возникает чувство, что операторы метода g1() не выполняются, но это не так. При добавлении методов в делегат и его выполнении выполняются все добавленные операторы за исключением операторов return. Эти операторы выполнятся только у последнего метода. Поэтому предыдущие методы могут влиять на результат только при наличии некоторых побочных эффектов.

Рассмотрим пример. Определим делегат

```
delegate int St(ref int x);
```

и функции, обладающие побочным эффектом,

```
static int t1(ref int x)
{
    x += 2;
    return x;
}
static int t2(ref int x)
{
    x += 3;
```

```
    return x;
}
static int t3(ref int x)
{
    x += 4;
    return x;
}
```

В процедуру `Main()` запишем код

```
St st1 = t1;
st1 += t2;
st1 += t3;
int a = 5;
int b = 5;
Console.WriteLine("a = {0}, st1(5) = {1}, a = {2}",
                a, st1(ref a), a);
Console.WriteLine("b = {0}, t3(5) = {1}",
                    b, t3(ref b));
```

При выполнении кода получим

```
a = 5, st1(5) = 14, a = 14
b = 5, t3(5) = 9
```

Проанализируем результат. При выполнении метода `t1()` имеем `a = 7`, при выполнении метода `t2()` имеем `a = 10`, при выполнении метода `t3()` имеем `a = 14`. Это значение и будет возвращено. Все это соответствует первой строке вывода. Вторая строка здесь предназначена для сравнения, что было бы, если бы мы выполняли отдельно только метод `t3()`.

Близким к рассмотренным здесь вопросам является объединение делегатов. Правомочным является запись

```
Del del2 = f1;
Del del3 = Test.f2;
Del del4 = del2 + del3;
del4();
```

дающий ожидаемый результат

```
Привет!
Как дела?
```

или

```
St st1 = t1;
St st2 = t2;
St st3 = t3;
St st4 = st1 + st2 + st3;
int a = 5;
Console.WriteLine("a = {0}, st4(5) = {1}, a = {2}",
                a, st4(ref a), a);
```

Здесь также получим

```
a = 5, st4(5) = 14, a = 14
```

§ 6.3. Использование делегатов при описании методов

Делегаты определяют некоторые типы, и эти типы можно использовать при описании методов. Сначала мы их будем использовать при определении типов параметров, т.е. метод в качестве параметра будет принимать переменную типа делегат.

Рассмотрим пример. Опишем метод

```
static void Operator(int a, int b, Fun f) =>
      Console.WriteLine("Первое число {0}, второе число
         {1}, результат {2}", a, b, f(a, b));
```

Как видим, этот метод имеет три параметра, два из них типа `int` и один типа делегата `Fun`. Выполняя операторы

```
Operator(4, 5, g1);
Operator(4, 5, g2);
```

получим

```
Первое число 4, второе число 5, результат 9
Первое число 4, второе число 5, результат 20
```

При первом выполнении метода `Operator()` в качестве значения третьего параметра использовался метод `g1()`, а при втором – метод `g2()`.

Сейчас рассмотрим пример, когда делегат определяет тип возвращаемого значения. Опишем метод

```
static Del dd(int k)
{
    switch (k)
    {
        case 1: return f1;
        case 2: return Test.f2;
        default: return new Test().f3;
    }
}
```

Эта функция возвращает значение типа делегата `Del` в зависимости от значения входного параметра. Выполним операторы

```
dd(1)();
dd(2)();
dd(6)();
```

и получим строки

```
Привет!
Как дела?
До свидания!
```

Заметим, что при вызове метода мы использовали две пары скобок. Первая пара относится самому методу, а вторая – делегату. Кроме того, мы функцию вызывали как процедуру. Это связано с тем, что функция возвращает делегат типа `void`.

Метод `dd()` мы могли бы использовать так:

```
Console.WriteLine("Введите число");
int z = int.Parse(Console.ReadLine());
dd(z)();
```

Теперь выводимая строка будет зависеть от введенного числа.

§ 6.4. Анонимные методы и лямбда-выражения

Иногда методы используются только для инициализации делегатов или добавления к ним, причем только один раз. В этом случае можно тело метода записать непосредственно в месте его применения, не используя

при этом название. Такие методы называются анонимными. Их описание имеет вид

```
delegate(параметры)
{
    тело
}
```

Если параметры отсутствуют, то пустые скобки можно опустить.

При выполнении операторов

```
Fun fun2 = delegate (int x, int y)
{
    return x - y;
};
Console.WriteLine(fun2(3, 4));
```

получим

```
-1
```

Приведем второй пример с использованием добавления методов:

```
Del del5 = f1;
del5 += delegate
{
    Console.WriteLine("Что нового?");
    Console.WriteLine("Будьте здоровы!");
};
del1();
```

Результат будет иметь вид

```
Привет!
Что нового?
Будьте здоровы!
```

Анонимный метод также может использоваться в качестве фактического параметра при вызове соответствующего метода. Примером может служить оператор

```
Operator(5, 3, delegate (int x, int y)
```

```
      { return x - y; });
```

при выполнении которого получим строку

```
Первое число 5, второе число 3, результат 2
```

Лямбда-выражения представляют упрощенную запись анонимных методов. По своей структуре напоминают сокращенную запись методов, и имеют вид

```
(параметры) => тело
```

Если тело состоит из одного оператора, то фигурные скобки можно опускать. Если тело состоит из одного оператора `return`, то слово `return` опускается. При наличии параметров можно не указывать их тип. Пустые скобки опускать нельзя.

Рассмотренные выше коды с анонимными методами, в случае лямбда-выражений примут вид, соответственно,

```
Fun fun2 =  (x, y) => x - y;
Console.WriteLine(fun2(3, 4));
```

```
Del del1 = f1;
del1 += () =>
{
    Console.WriteLine("Что нового?");
    Console.WriteLine("Будьте здоровы!");
};
del1();
```

и

```
Operator(5, 3,  (int x, int y) => x - y);
```

При этом мы получим те же самые результаты.

Анонимные методы и лямбда-выражения, как и обычные методы, могут использоваться в качестве возвращаемых значений функций. Для примера приведем следующую функцию:

```
static Del dd2(int k)
{
```

```
    switch (k)
    {
        case 1: return delegate
           { Console.WriteLine("Благодарю Вас!"); };
        case 2: return ()=>
             Console.WriteLine("Всего хорошего!");
        default: return f1;
    }
}
```

Здесь в качестве возвращаемых значений использованы анонимный метод, лямбда-выражение и обычный метод. Все они соответствуют делегату `Del`.

При выполнении фрагмента кода

```
dd2(3)();
dd2(2)();
dd2(1)();
```

получим строки

```
Привет!
Всего хорошего!
Благодарю Вас!
```

§ 6.5. Делегаты как поля классов

Выше мы при описании функций использовали делегаты для определения типа параметров или выводимого значения. Можно делегаты использовать и в описании класса для определения типов полей. Это может позволить создать очень эффективные коды. Дело в том, что в большинстве рассмотренных выше примеров применение делегатов и описание соответствующих методов происходило в пределах одного класса. В принципе, можно было непосредственно использовать методы, не привлекая сюда делегаты. Совсем другое дело, когда применение делегатов осуществляется в одном классе, а описание методов и инициализация делегатов в другом классе. Разрабатывая первый класс, мы можем и не знать, какие непосредственно методы потом будут использоваться. Например, мы можем указать, что надо вывести сообщение с определенным текстом, но мы не знаем, куда конкретно будет выводиться сообщение. Это может быть консольное окно, текстовое окно Windows-приложения или дисплей мобильного устройства. Это будет

определяться при написании самой программы, и используемых вариантов может быть несколько. Поэтому в первом классе нам удобно использовать соответствующий делегат.

Для иллюстрации сказанного, рассмотрим пример. Предположим, что нам нужно создать класс для хранения счетов в банке с двумя методами: для пополнения счета и снятия денег со счета. Причем требуется выводить соответствующее сообщение. Необходимо учесть, что при снятии со счета имеющихся денег может не хватить. Вначале мы для вывода сообщений будем использовать консоль. С учетом сказанного класс можно описать так:

```
class Account
{
    private int sum;
    public Account(int sum)
    {
        this.sum = sum;
    }
    public void Add(int s)
    {
        sum += s;
        Console.WriteLine("Пополнено, остаток {0}",sum);
    }
    public void Sub(int s)
    {
        if(s <= sum)
        {
            sum -= s;
            Console.WriteLine("Снято, остаток {0}", sum);
        }
        else
        {
        Console.WriteLine("Средств недостаточно,
              остаток {0}", sum);
        }
    }
}
```

Здесь мы описали поле для хранения остатка средств, конструктор с параметром для инициализации поля и два метода. Первый метод предназначен для пополнения счета, а второй - для снятия денег. Причем

при снятии осуществляется проверка наличия средств на счете для выполнения снятия. Каждый раз выводится сообщение на консоль.

В процедуру `Main()` запишем код

```
Account ac1 = new Account(100);
ac1.Sub(50);
ac1.Add(20);
ac1.Sub(80);
ac1.Add(30);
```

При его выполнении получим

```
Снято, остаток 50
Пополнено, остаток 70
Средств недостаточно, остаток 70
Пополнено, остаток 100
```

Все работает, но только в случае консольного приложения. Если же мы захотим создать приложение любого другого типа, то класс `Account` использовать не сможем. Для решения указанной проблемы этот класс несколько модернизируем, и соответствующий фрагмент кода приведем к виду

```
delegate void Info(string s);
class Account
{
    private int sum;
    private Info sob;
    public void Registr(Info x) => sob = x;
    public Account(int sum)
    {
        this.sum = sum;
    }
    public void Add(int s)
    {
        sum += s;
        sob(string.Format("Пополнено, остаток {0}", sum));
    }
    public void Sub(int s)
    {
        if(s <= sum)
```

```
        {
            sum -= s;
            sob(string.Format("Снято, остаток {0}", sum));
        }
        else
        {
            sob(string.Format("Средств недостаточно,
                 остаток {0}", sum));
        }
    }
}
```

Сравним две версии кода. Здесь мы в код добавили описания делегата `Info`, нового поля типа `Info` и метода для инициализации этого поля. Кроме того, вызовы метода `Console.WriteLine()` заменили вызовами делегата добавленного поля с тем же параметром. Для формирования строки нам пришлось применить метод `string.Format()`. Для процедуры же `Console.WriteLine()` этот метод применяется по умолчанию.

Теперь рассмотрим изменения, которые следует внести в класс `Program`. Очевидно, нам в любом случае нужно инициализировать поле типа делегата. Для этого можно использовать либо обычный метод, либо анонимный метод, либо лямбда-выражение. Причем, как обычно, можно добавлять несколько методов. Мы в класс `Program` добавим метод

```
static void Rez(string z) =>
           Console.WriteLine("Обычный метод: " + z);
```

а в процедуру `Main()` запишем операторы

```
Account ac1 = new Account(100);

Info in1 = Rez;
in1 += delegate(string z)
       {
           Console.WriteLine("Анонимный метод: " + z);
       };
in1 += (z) =>
{
    Console.WriteLine("Лямбла-выражение: " + z);
    Console.WriteLine();
```

```
}
ac1.Registr(in1);

ac1.Sub(50);
ac1.Add(20);
ac1.Sub(80);
ac1.Add(30);
```

Здесь у нас между созданием нового счета и действиями с ним появился блок операторов, предназначенных для инициализации поля типа делегат. При этом мы использовали три варианта добавления методов с некоторым отличием в выводимой строке. У нас при каждом действии со счетом будут выводиться три строки на консоль. Естественно, на практике вывод осуществлялся бы в три разных устройства, например, в принтер, на электронную почту и SMS-сообщение.

Выполнение программы приведет к результату

```
Обычный метод: Снято, остаток 50
Анонимный метод: Снято, остаток 50
Лямбда-выражение: Снято, остаток 50

Обычный метод: Пополнено, остаток 70
Анонимный метод: Пополнено, остаток 70
Лямбда-выражение: Пополнено, остаток 70

Обычный метод: Средств недостаточно, остаток 70
Анонимный метод: Средств недостаточно, остаток 70
Лямбда-выражение: Средств недостаточно, остаток 70

Обычный метод: Пополнено, остаток 100
Анонимный метод: Пополнено, остаток 100
Лямбда-выражение: Пополнено, остаток 100
```

Как видим, возможности использования класса `Account` расширились.

§ 6.6. События

В классе `Account` мы для инициализации поля типа делегат использовали специальный, созданный для этого, метод. Конечно, мы могли бы установить для этого поля модификатор доступа `public` и

получить доступ к нему из других классов, но, как мы знаем, при этом возникают проблемы безопасности. Для полей других типов решение данной проблемы осуществляется за счет использования свойств, которые представляют собой инструмент доступа к полям. В случае же полей типа делегат эту функцию выполняют события.

Класс, который порождает (отправляет) событие, называется издателем, а классы, обрабатывающие (принимающие) событие, называются подписчиками. Соответственно, на одно и то же событие могут подписываться несколько подписчиков.

Рассмотрим принципы описания и использования событий. Заголовок события аналогичен заголовку свойства с добавлением слова `event`. Как и для свойств, существует сокращенная запись и развернутая.

Первая форма выглядит так:

```
public event тип_делегата имя;
```

В этом случае определяется одноименное поле указанного типа делегата, и событие обеспечивает доступ к нему. Никакой дополнительный функционал при этом не имеется. Используются события по аналогии со свойствами только для записи. При этом используются операции `+=` и `-=`. Первая из операций называется подпиской.

В качестве примера немного отредактируем класс `Account`, а именно, строки

```
private Info sob;
public void Registr(Info x) => sob = x;
```

заменим одним

```
public event Info sob;
```

В процедуру же `Main()` запишем код

```
Account ac1 = new Account(100);
ac1.sob += Rez;
ac1.sob += delegate (string z)
{
    Console.WriteLine("Анонимный метод: " + z);
};
ac1.sob += (z) =>
{
    Console.WriteLine("Лямбда-выражение: " + z);
```

```
    Console.WriteLine();
};

ac1.Sub(50);
ac1.Add(20);
ac1.Sub(80);
ac1.Add(30);
```

При выполнении кода получим результат, совпадающий с результатом от предыдущей версии кода.

Сейчас рассмотрим развернутую запись события. В этом случае описание события содержит два блока: `add` и `remove`. Первый блок выполняется при попытке добавить делегату некоторый метод, а второй – при попытке удалить из делегата некоторый метод. Само добавление и удаление выполняется, соответственно, операторами вида

```
поле += value;
```

и

```
поле -= value;
```

Кроме данных действий, в этих блоках можно, например, организовать доступ по пароли, выводить имя добавляемого или удаляемого метода, выводить некоторую другую информацию.

Рассмотрим пример. Опишем класс `AccountSob`, аналогичный классу `Account`, только при попытке добавления или удаления методов будем запрашивать пароль и выводить имя самого метода. Код имеет вид

```
class AccountSob
{
    private int sum;
    private Info sob;
    public event Info Sob
    {
        add
        {
            Console.WriteLine("Введите пароль");
            string pw = Console.ReadLine();
            if (pw == "123")
            {
                sob += value;
```

```
                Console.WriteLine("Добавлен метод  " +
                   value.Method.Name);
            }
        }

        remove
        {
            Console.WriteLine("Введите пароль");
            string pw = Console.ReadLine();
            if (pw == "123")
            {
                sob -= value;
                Console.WriteLine("Удален метод  " +
                     value.Method.Name);
            }
        }
    }
    public AccountSob(int sum)
    {
        this.sum = sum;
    }
    public void Add(int s)
    {
        sum += s;
        sob(string.Format("Пополнено, остаток {0}", sum));
    }
    public void Sub(int s)
    {
        if (s <= sum)
        {
            sum -= s;
            sob(string.Format("Снято, остаток {0}", sum));
        }
        else
        {
            sob(string.Format("Средств недостаточно,
                остаток {0}", sum));
        }
    }
}
```

В процедуру `Main()` запишем операторы

```
AccountSob ac2 = new AccountSob(100);
ac2.Sob += Rez;
ac2.Sub(50);
ac2.Sob += delegate (string z)
           {
              Console.WriteLine("Анонимный метод: " + z);
           };
ac2.Add(20);
ac2.Sob += Rez;
ac2.Sob += (z) =>
           {
              Console.WriteLine("Лямбла-выражение: " + z);
              Console.WriteLine();
           };
ac2.Sub(80);
ac2.Sob -= Rez;
ac2.Add(30);
```

При их выполнении получим строки

```
Введите пароль
123
Добавлен метод Rez
Обычный метод: Снято, остаток 50
Введите пароль
123
Добавлен метод <Main>b__1_0
Обычный метод: Пополнено, остаток 70
Анонимный метод: Пополнено, остаток 70
Введите пароль
122
Введите пароль
123
Добавлен метод <Main>b__1_1
Обычный метод: Средств недостаточно, остаток 70
Анонимный метод: Средств недостаточно, остаток 70
Лямбда-выражение: Средств недостаточно, остаток 70

Введите пароль
```

```
123
Удален метод Rez
Анонимный метод: Пополнено, остаток 100
Лямбда-выражение: Пополнено, остаток 100
```

Как видим, пароль запрашивается при каждом обращении к событию, причем при третьем запросе мы неправильно ввели пароль. Поэтому операция добавления была проигнорирована и, метод `Rez` не был повторно добавлен. Можно также обратить внимание, какие имена были выведены для анонимного метода и лямбда-выражения.

Ранее мы сказали, что событие – это инструмент доступа к полю типа делегат, но обычно в компьютерных науках под наступлением события понимают совершение некоторого действия, например, нажатие кнопки, изменение положения, истечение определенного промежутка времени и т.п. Поясним, как согласуются эти два понятия. Действительно, наступление некоторого события предполагает изменение значения определенного поля соответствующего экземпляра класса. Изменение этого значения и сопровождается вызовом делегата, которое приводит к выполнению заданного кода. Этот код называется обработчиком события. Нажимаем на кнопку. Значение поля, определяющее нажатие, меняется на `true`, и выполняется некоторая процедура из класса подписчика. В нашем же случае, совершение события связано попыткой изменения остатка на счете. Обработчик события выводит соответствующую информацию.

§ 6.7. Делегат `EventHandler`

При описании события мы использовали делегат типа `Info`, описанный в нашем проекте. Это в принципе допустимо, и мы получаем рабочую программу. Однако считается признаком хорошего тона использование при создании событий встроенного делегата `EventHandler`. Это делегат типа `void` с двумя параметрами. Первый параметр должен передать подписчику информацию, связанную с объектом, вызвавшим событие, а второй – должен иметь тип наследника класса `EventArgs` и передать информацию о причинах наступления события.

Рассмотрим пример со счетом. Начнем с описания дочернего класса для `EventArg`. Сам этот класс можно рассматривать как пустой контейнер. Нужно добавить необходимые члены. Мы добавим два поля для хранения, соответственно, остатка до совершения операции и после нее. При этом воспользуемся автоматическими свойствами для чтения. Также добавим конструктор для инициализации этих полей. Код примет вид

```
class AccountEventArgs:EventArgs
{
    public int OldSum { get; }
    public int NewSum { get; }
    public AccountEventArgs(int oldSum,int newSum)
    {
        OldSum = oldSum;
        NewSum = newSum;
    }
}
```

Перейдем к классу `AccountEvent`. Его код запишем так:

```
class AccountEvent
{
    private int sum;
    public string Name { get; }
    private EventHandler sob;
    public event EventHandler Sob
    {
        add
        {
            Console.WriteLine("Введите пароль");
            string pw = Console.ReadLine();
            if (pw == "123")
            {
                sob += value;
                Console.WriteLine("Добавлен метод  " +
                    value.Method.Name);
            }
        }

        remove
        {
            Console.WriteLine("Введите пароль");
            string pw = Console.ReadLine();
            if (pw == "123")
            {
                sob -= value;
                Console.WriteLine("Удален метод  " +
                     value.Method.Name);
```

```
            }
        }
    }

    public AccountEvent(string name,int sum)
    {
        this.sum = sum;
        Name = name;
    }
    public void Add(int s)
    {
        int old = sum;
        sum += s;
        sob(this, new AccountEventArgs( old,  sum));
    }
    public void Sub(int s)
    {
        if (s <= sum)
        {
            int old = sum;
            sum -= s;
            sob(this, new AccountEventArgs(old, sum));
        }
    }
}
```

Для вывода информации об объекте будем использовать фамилию клиента. Поэтому добавили соответствующее поле при помощи автоматического свойства. Соответственно изменился и конструктор. В описании события изменений нет. Наиболее серьезные изменения имеются в методах для добавления и снятия средств. Здесь мы сохраняем два значения остатка и на их основе создаем экземпляр класса `AccountEventArgs`. Этот объект вместе с используемым экземпляром самого класса `AccountEvent` передаются при вызове делегата.

Теперь перейдем к классу `Program`, который является подписчиком к событию. Мы здесь не можем в качестве обработчика события использовать метод `Rez()`. Так как он не соответствует типу делегата. Опишем подходящий метод, например, так:

```
static void Event1(object a, EventArgs e)
{
```

```
    string name = ((AccountEvent)a).Name;
    int oldSum = ((AccountEventArgs)e).OldSum;
    int newSum = ((AccountEventArgs)e).NewSum;

    Console.WriteLine("Клиент {0} было {1} стало {2}",
          name, oldSum, newSum);
}
```

Здесь мы применили операции приведения объектов к соответствующему типу, а затем использовали их свойства.

В процедуру `Main()` запишем операторы

```
AccountEvent acc2 = new AccountEvent("Иванов", 30);
acc2.Sob += Event1;
acc2.Add(50);
acc2.Sub(100);
acc2.Sub(60);
```

При их выполнении получим

```
Введите пароль
123
Добавлен метод Event1
Клиент Иванов было 30 стало 80
Клиент Иванов было 80 стало 20
```

По результату мы видим, что первая операция по снятию не прошла. Так как остаток был меньше запрошенной суммы.

§ 6.8. Встроенные делегаты

Выше мы для создания события использовали делегат `EventHandler`, который является встроенным. Поэтому не было необходимости описывать его. Имеются и другие встроенные делегаты, которые могут применяться в различных ситуациях. Рассмотрим некоторые из них.

Делегат `Action` имеет тип `void` и может содержать до шестнадцати входных параметров обобщенного типа. Для примера, в случаях отсутствия параметров и одного параметра описания делегата имеют, соответственно, вид

```
public delegate void Action();
```

```
public delegate void Action<T>(T x);
```

Вспоминая описания использованных нами делегатов, видим, что делегаты `Del` и `Info` соответствуют приведенным случаям. Поэтому мы, например, в классе `Program` вместо использованного нами оператора

```
Del del1 = f1;
```

могли бы сразу записать

```
Action del1 = f1;
```

вообще не вводя делегат `Del`. Или в классе Account вместо оператора

```
public event Info sob;
```

могли бы написать

```
public event Action<string> sob;
```

без изменения получаемых результатов. В этом случае описание делегата `Info` не требуется.

Следующим является делегат `Func`. Этот делегат также может иметь до шестнадцати входных параметров, но в отличие от предыдущего делегата он возвращает результат. Возвращаемому результату соответствует последний универсальный параметр.

Мы в классе `Program` использовали делегат `Fun`, он соответствует `Func<int,int,int>`. Поэтому мы вместо оператора

```
Fun fun1 = g1;
```

могли бы написать

```
Func<int,int,int> fun1 = g1;
```

Отметим, что мы в этом же классе использовали делегат

```
T Pr<T, P, K>(P x, K y)
```

который равносилен следующему

```
T Func<P, K, T>(P x, K y)
```

Отличие состоит в том, что для использования первый делегат описывать нужно, а второй – нет.

Также часто используется делегат `Predicate`. Этот делегат имеет один входной параметр обобщенного типа и возвращает значение типа `bool`. Они, как правило, применяются для проверки выполнения определенного условия. В качестве выходного результата возвращается значение `true`, если условие выполнено, и `false`, если не выполнено. Приведем пример кода

```
Predicate<int> isPositive = (int x) => x > 0;
Console.WriteLine(isPositive(20));
Console.WriteLine(isPositive(-20));
```

В данном случае возвращается `true` или `false` в зависимости от того, введенное число больше нуля или нет. Получим результат

```
True
False
```

Рассмотренные делегаты часто используются как параметры различных методов.

На этом мы изложение материала завершим.

Хайруллин Равиль Сагитович

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ

Учебное пособие

Редактор

Издательство
Казанского государственного архитектурно-строительного университета
Подписано в печать ******* Формат 60x84/16
Заказ № *** Печать ризографическая Усл.-печ.л. *,*
Тираж ** экз. Бумага офсетная № 1 Ус.-изд. Л. *,*

Отпечатано в полиграфическом секторе
Издательства КГАСУ
420043, г.Казань, ул. Зеленая, д.1